CS 598 Practical Statistical Learning

Alexandra Chronopoulou

2026-08-21

Contents

Course Information

Course Description

This course provides an introduction to modern techniques for statistical analysis of complex and massive data. Examples of these include model selection for regression, classification, nonparametric models such as splines and kernel models, regularization, dimension reduction, and clustering analysis. Applications are discussed as well as computation and theoretical foundations.

Course Prerequisites

Knowledge of basic multivariate calculus, statistical inference, and linear algebra. You should be comfortable with the following concepts: probability distribution functions, expectations, conditional distributions, likelihood functions, random samples, estimators, and linear regression models.

Course Learning Outcomes

By the end of the course, you will be able to:

- Use a broad range of methods and techniques in machine learning.
- Have a deeper understanding of major algorithms and techniques in machine learning.
- Build analytics pipelines for regression problems.
- Build analytics pipelines for classification problems.
- Build analytics pipelines for recommendation problems.

Textbook and Readings

There is no required textbook for this course.

Recommended resources for a statistical approach to machine learning are:

- *An Introduction to Statistical Learning with Applications in R* (basic)
- *An Introduction to Statistical Learning with Applications in Python* (basic)

- *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (more advanced)

You may also want to view the Data School YouTube videos associated with the first book as an additional resource.

The detailed syllabus for the course can be found on Coursera.

Chapter 1

Introduction to Statistical Learning

Statistical Learning is a field at the intersection of statistics, computer science, and data science that focuses on developing and understanding models that can *learn* from data. While statistics has long dealt with model building, validation, and prediction, the dramatic growth in the volume, complexity, and dimensionality of data has strained many classical techniques. The introduction of sophisticated algorithms, data-management tools, and optimization methods from computer science has enabled more powerful approaches suited to modern data regimes.

In the supervised setting, learning from data typically means that, given an outcome (quantitative or categorical) and a (usually large) set of features, we construct a **prediction model** (also called a *learner*) that can accurately forecast the outcome for new, previously unseen observations.

We can separate statistical learning problems in *two types*:

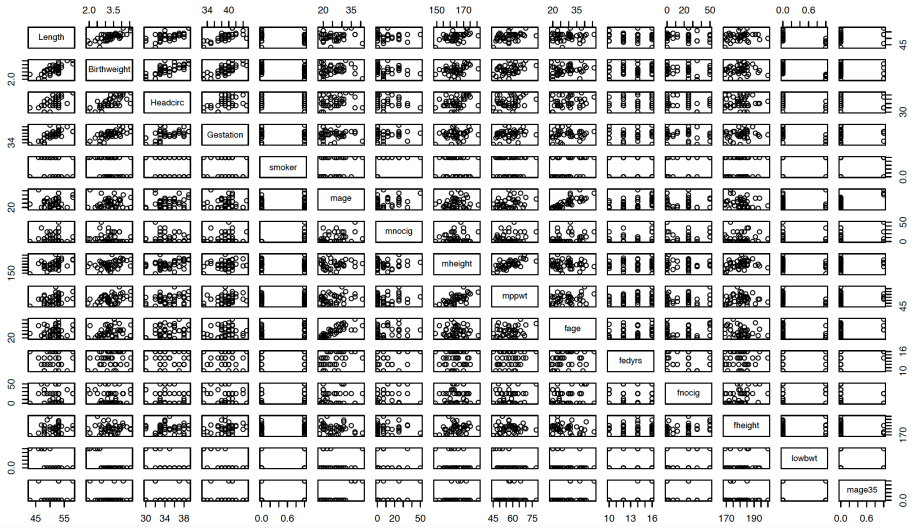
- **Supervised Learning**: when an outcome variable is present to guide the learning process. The two primary examples are:
  - *Regression*: Response is quantitative (continuous or discrete numeric).
  - *Classification*: Response is categorical (binary or multi-class).
- **Unsupervised Learning**:when an outcome variable is not present to guide the learning process. The goal is to discover latent structure or patterns in the data (e.g., clustering, association rules, hidden Markov models, etc.).

1.1 Examples of Statistical Learning Problems

Birthweight Data: A Regression Example

This dataset comes from a study whose goal is to predict the birth weight of a newborn from a collection of baby and parental characteristics (gestation length, length, head circumference, mother's and father's height, mother's weight, parental smoking status, etc.). The study focuses particularly on babies born prematurely (before 40 weeks of gestation).

The scatterplot matrix below shows the pairwise relationships among the variables in the dataset:



Note that three of the predictors (`smoker`, `lowbwt`, and `mage35`) are categorical.

This is a classic example of a supervised regression problem and a natural setting in which a multiple linear regression (MLR) model can be considered. If the MLR model fits well and the key assumptions are reasonably satisfied, it can provide useful predictions of birth weight for new observations.

Trees and Shrubs Data: A Binary Classification Example

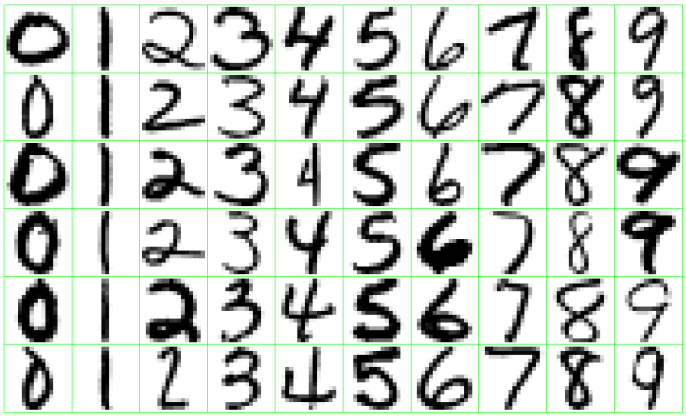
In this problem, the goal is to predict whether a woody plant common in the Black Forest region of southwestern Germany is a tree or a shrub (Lederer). A sample of the data is shown below:

Name	Height	Diameter	GrowthRate	Longevity	Type
Silver Fir	55.0	1.50	0.5	500	Tree
European Beech	40.0	1.00	0.5	200	Tree
Common Hazel	5.0	0.10	0.5	70	Shrub
Red Raspberry	1.5	0.01	0.4	10	Shrub
European Spruce	50.0	1.20	0.5	600	Tree

This is a binary classification problem in which we want to use a set of characteristics to predict the class label (tree or shrub).

Handwritten Digits: A Multiclass Classification Example

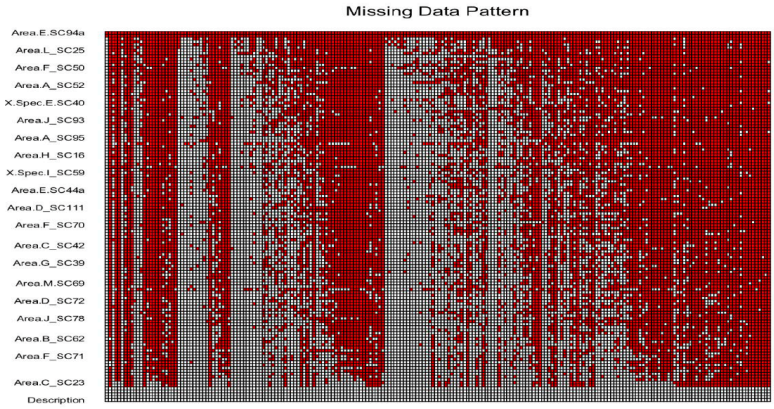
This is a classic multiclass classification problem in which the goal is to predict the handwritten digit $(0, \ldots, 9)$ appearing on a postal envelope, based on a 16×16 eight-bit grayscale image (ESL). The practical challenge is to keep the error rate sufficiently low so that mail is not misdirected.



Proteomics Data: A Clustering Example

Proteomics is the large-scale study of proteins. Data of this type are typically generated through laboratory processes such as protein purification or mass spectrometry (a technique that measures the mass-to-charge ratio of ions).

The figure below shows an expression matrix for 437 proteins, of which only 101 are of primary interest (Romanova et al.). A common challenge with such data is the large amount of missing values (shown as grey pixels). The goal is to cluster the proteins into coherent groups for further biological analysis.



These examples illustrate the variety of problems that arise in statistical learning. The birthweight data is a typical supervised regression problem in which we aim to predict a quantitative outcome. The trees-and-shrubs data is a binary classification task, while the handwritten-digit data is a multiclass classification problem. Finally, the proteomics data is an unsupervised learning example whose goal is to uncover latent structure through clustering. Together they show how statistical learning methods can be applied across different types of outcomes and data structures, whether the aim is *prediction* or the *discovery of patterns*.

1.2 Supervised Learning Framework

In this class, our main focus is **supervised learning**. This means that we observe **target** (*outcome*, *response*) variable Y that we wish to predict using a set of *features* (*predictors*), X. We assume that the relationship between the response Y and the features is given by a function f that is known up to a parameter vector w:

$$\mathbf{Y} = f(\mathbf{X}, w) + varepsilon$$

When f is fully known except for the value of w (for example when f is linear), then learning problem reduces to **estimating** w. To do so, we collect a **training sample** $\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^n$ and minimize an **objective** function_with respect to w:

$$F(w) = \sum_{i=1}^{n} loss\Big(y_i,\ f(\mathbf{x}_i, w)\Big)$$

The **loss** function measures the discrepancy between the model prediction $f(\mathbf{x}_i, w)$ and the observed outcome $\mathbf{y}_i$. The choice of loss is made by the practitioner; the most common examples are:

- **Squared Error Loss**

$$\text{loss} = \Big[y_i - f(x_i; w)\Big]^2$$

- **Absolute Error Loss**

$$\text{loss} = \Big|y_i - f(x_i; w)\Big|$$

- **Log Loss** (binary classification)

$$\text{loss} = -y_i \log f(x_i; w) - (1 - y_i) \log(1 - f(x_i; w))$$

- **0-1 Loss**

$$\text{loss} = \mathbf{1}_{\{y_i \neq \hat{y}_i\}} \quad \text{where} \quad \hat{y}_i = f(x_i; w)$$

Depending on the chosen loss, the minimizer of $F(w)$ may or may not admit a closed-form expression. When a closed form is unavailable we resort to numerical optimization algorithms. In many cases gradient-based methods (in particular gradient descent and its variants) can be used to obtain a good solution.

1.3 Why Statistical Learning is Challenging

In the framework of the previous section we reduced the learning problem to the estimation of a parameter vector w, under the assumption that the true function f is known up to this parameter. In practice, however, we rarely know the exact form of f. We must therefore *approximate* it by some function $\widehat{f}$. This approximation introduces an *additional source of error*.

Moreover, the objective function we minimize is computed on a *training* sample. While this quantifies how well the model fits the observed data, the real goal of statistical learning is good performance on **new**, **unseen** data. In other words, we aim to minimize the **test error** (also called generalization error), not the training error.

Statistical learning is therefore challenging for several reasons:

- The training error typically underestimates the test/generalization error.
- A model may perform well on the training data yet poorly on future data because of overfitting. The number of parameters required to approximate f can be large—sometimes larger than the number of available observations.
- As the number of model parameters grows, the test error often increases substantially.

Two simple illustrations make these points concrete:

1. **Classification: 1-Nearest Neighbour**
   The one-nearest-neighbour classifier achieves zero training error (it predicts every training point perfectly) but typically performs poorly on test data. An instructive demonstration of how increasing dimensionality affects the performance of linear classifiers can be found here.

2. **Regression: the p versus n problem**
   Consider the linear model

   $$\mathbf{y}_{n\times 1} = \mathbf{X}_{n\times p}\mathbf{w}_{p\times 1} + \varepsilon,$$

   where $\mathbf{y}$ is the response vector, $\mathbf{X}$ is the design matrix with p predictors, and $\mathbf{w}$ contains the coefficients to be estimated.

   - When $p < n$ we have more equations than parameters; classical regression methods usually work well.
   - When $p = n$ we have exactly as many equations as parameters and can achieve a perfect fit on the training data.
   - When $p > n$ we have fewer equations than parameters. In this high-dimensional regime classical methods break down and specialized techniques are required.

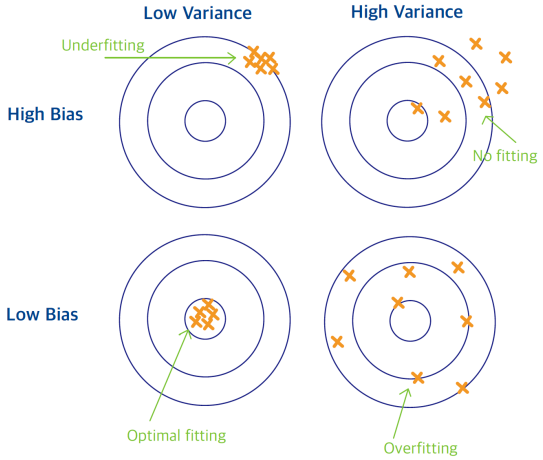
In summary, statistical learning is challenging because we must approximate an unknown function f, control the gap between training and test error, and often operate in regimes where the number of parameters is large relative to the sample size. Successfully addressing these issues is essential for building models that generalize well to new data.

1.4 Bias-Variance Trade-Off

We have already noted that *two sources of error* affect the quality of our predictions. In statistical learning we must balance:

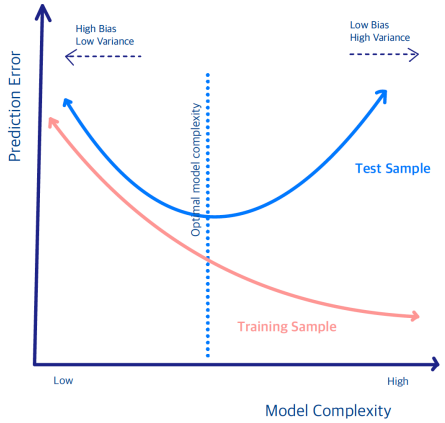
- **Bias**: the systematic difference between the average of our estimated function $\widehat{f}$ and the true underlying function f. High bias typically leads to under-fitting and an inaccurate model.
- **Variance**: the amount by which $\widehat{f}$ would change if we estimated it using a different training sample. High variance typically leads to over-fitting and an unreliable model.

A helpful analogy is to think of the learning problem as a **target** whose *center represents the true model.*



How close the fitted values lie to the center is measured by bias. If the points systematically miss the center, the model is a poor approximation of f. Increasing the sample size will not correct a large bias. The spread of the fitted values around their average is measured by variance. When the points are tightly clustered the variance is low; when they are widely scattered the variance is high.

Ideally we would like to reduce both bias and variance simultaneously. In practice this is not possible, so we must trade one against the other. Remembering that our ultimate goal is to minimize the generalization (test) error, not the training error, we observe the following typical behavior:



- When the model is highly complex (many parameters), the training error tends to decrease. However, the model often over-fits the training data and therefore exhibits high variance on new data, resulting in poor generalization.

- When the model is very simple (few parameters), it tends to under-fit the data. This produces large bias and, again, poor generalization performance.

In this class we will discuss:

(i) flexible modeling techniques that help reduce bias, and

(ii) practical strategies for achieving a good bias-variance trade-off.

Two important approaches that we will study in detail are:

- **Regularization**: constrain the parameters to a lower-dimensional space that is determined adaptively by the data.
- **Ensemble**: average many low-bias, high-variance models; the averaging step reduces variance while largely preserving the low bias.

1.5 Two Toy Examples: k-NN vs. Linear Regression

Before we conclude this introductory chapter, we examine two simple supervised learning methods:

(i) k-Nearest Neighbors (k-NN)

(ii) Linear regression

We will compare their behavior on simple examples and use them to illustrate the bias-variance trade-off.

1.5.1 k-Nearest Neighbors

In the **k-nearest neighbors** (k-NN) method we predict the response at a point $\mathbf{x}$ by using the k training observations that are closest to $\mathbf{x}$. The k-NN fit is

$$\hat{\mathbf{y}}(\mathbf{x}) = \frac{1}{k} \sum_{x_i \in N_k(\mathbf{x})} y_i$$

where $N_k(\mathbf{x})$ denotes the neighborhood of $\mathbf{x}$ consisting of the k nearest training points.

- In **regression**, $\hat{y}(\mathbf{x})$ is a local average of the neighboring responses.

- In **classification**, $\hat{y}(\mathbf{x})$ is the majority class (or the class probability) within the neighborhood.

Two practical challenges that arise are choosing the neighborhood size k and choosing the distance metric that defines the neighborhood.

The value of k directly controls model complexity, which is roughly proportional to n/k.

- When $k = 1$ the prediction at each training point x_i is exactly y_i, yielding zero training error.

- When $k = n$ every neighborhood contains the entire training sample, so the prediction is constant for all $\mathbf{x}$.

The "default" metric is Euclidean distance

$$d(\mathbf{x}, \tilde{\mathbf{x}}) = \sum_{j=1}^{p} w_j (x_j - \tilde{x}_j)^2,$$

where the weights w_j may be learned from the data.

1.5.2 Linear Regression

Given an input vector $\mathbf{x}^\top = (x_1, \ldots, x_p)$, we approximate the response by a linear function

$$f(\mathbf{x}) \approx \beta_0 + \sum_{j=1}^{p} x_j \beta_j$$

The coefficients β_j are estimated by ordinary least squares (OLS), i.e., by minimizing the residual sum of squares (*objective function*)

$$\min_{\beta_0, \ldots, \beta_p} \sum_{i=1}^{n} \Big(y_i - \beta_0 - x_{i1}\beta_1 - \ldots - x_{ip}\beta_p \Big)^2$$

Under standard conditions the solution has a closed form, and the fitted value at the i-th observation is

$$\hat{y}_i = \hat{y}(x_i) = x_i^T \hat{\beta}$$

We postpone the algebraic details until Week 2.

Linear Regression in a Classification Setting

Linear regression can also be applied to binary classification problems in which $Y \in \{0, 1\}$. One simply predicts `class 1` when the fitted value $\hat{f}(\mathbf{x})$ exceeds 0.5 and `class 0` otherwise.

This approach has well-known drawbacks: the squared-error loss is not a natural measure for binary outcomes, and a linear function can produce fitted values outside the interval $[0, 1]$. For these reasons logistic regression is the preferred linear method for classification. It models the log-odds

$$\log \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} \approx \beta_0 + \sum_{j=1}^{p} x_j \beta_j$$

We will return to logistic regression in Week 8.

1.5.3 A Simulated (Binary) Classification Example

Consider a response variable G that takes two values (0 = BLUE or 1 = ORANGE). We generate 200 observations (100 in each class) and compare linear regression and k-nearest neighbors **as classifiers**.

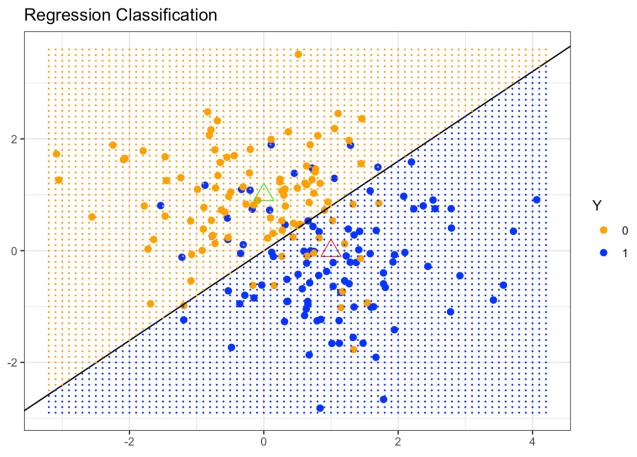
The code used to produce the simulation and all figures is available in R and Python.

Linear Regression Classifier

We first fit a linear regression model, treating the binary response as if it were continuous. The continuous fitted values $\hat{Y}$ are then converted to class predictions $\hat{G}$ by the simple rule

$$\hat{G} = \begin{cases} \textit{Blue}, \text{ if } \hat{Y} > 0.5 \\ \textit{Orange}, \text{ otherwise} \end{cases}$$

Because the problem is two-dimensional, the decision boundary is a *straight line*.



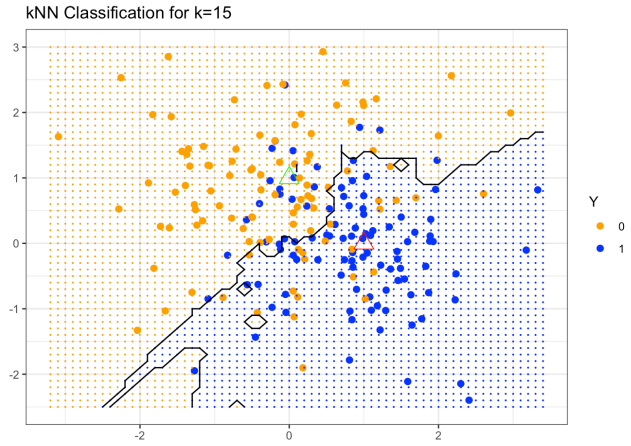
The fitted boundary (black line in the figure below) is the set of points satisfying $\mathbf{x}^\top \hat{\beta} = 0.5$. In the simulation the true blue region lies above this line and the orange region below it. Many points on both sides of the boundary are nevertheless misclassified. The linear decision boundary is smooth but rigid; it cannot adapt to local structure in the data.

k-Nearest-Neighbor Classifier

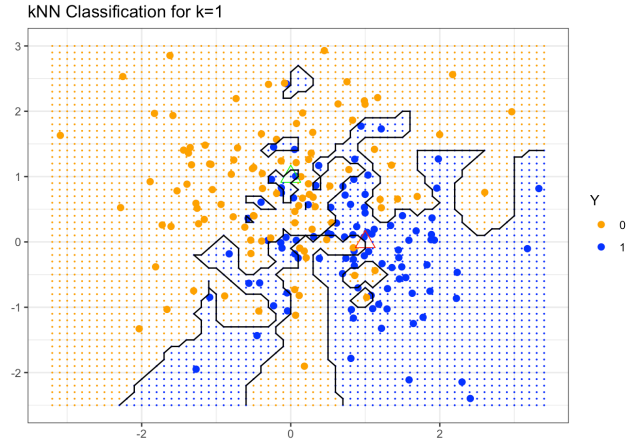
We now apply k-nearest neighbors to the same data. Using $k = 15$, $\hat{Y}(\mathbf{x})$ is the proportion of blue observations among the 15 nearest neighbors of $\mathbf{x}$. Class labels are again assigned by the 0.5 threshold:

$$\hat{G} = \begin{cases} \textit{Blue}, \text{ if } \hat{Y} > 0.5 \\ \textit{Orange}, \text{ otherwise} \end{cases}$$

(The prediction is therefore a majority vote among the 15 neighbors.)



The resulting decision boundary is far more irregular and follows local clusters of blue and orange points. Consequently the number of misclassifications on the training data is smaller than with linear regression. If we take the extreme case $k = 1$, each point is classified by its single nearest neighbor:

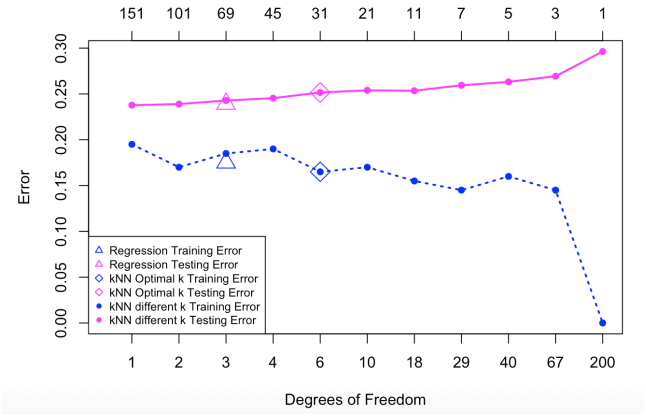


The boundary becomes extremely rough and the training misclassification rate drops nearly to zero. Whether this is desirable, however, can be judged only by examining the *generalization error*.

Training versus Test Error

Up to this point we have evaluated both methods on the same data that were used to fit them. A method such as 1-NN therefore appears to have zero error. Fair comparison requires an independent test set. The figure below shows misclassification error on a test set of 10,000 observations as a function of model degrees of freedom. Several k-NN fits (varying k) are compared with the linear

regression fit.



The magenta curve is the test error and the blue curve is the training error of k-NN. A 5-fold cross-validation procedure selected the optimal k (marked by a diamond). Linear regression results appear as the magenta and blue triangles at 3 degrees of freedom (the dimension of the fitted linear model). Linear regression has only a few parameters and therefore relatively low variance, but the linearity assumption is restrictive for this problem and produces high bias.

In contrast, k-NN makes almost no assumption about the shape of the decision boundary (apart from a mild local-smoothness condition). This flexibility yields low bias but high variance; the extreme case $k = 1$ corresponds to roughly n parameters and severely overfits. It can be shown that k-NN is consistent when $k, n \to \infty$ with $k/n \to 0$. The price of that consistency is the high variance that appears when the effective number of parameters becomes large.

Chapter 2

Linear Regression Models

A regression model is a model for the **conditional expectation** of the response/outcome variable given a set of predictors/features. A linear regression model specifically assumes that $E(Y \mid \mathbf{X} = \mathbf{x})$ is a linear function of the inputs $X_1, X_2, \ldots, X_p$, i.e.,

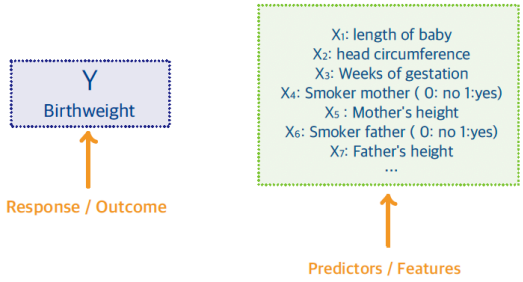
$$E(Y|\mathbf{X} = \mathbf{x}) \sim \beta_0 + \beta_1 X_1 + ... + \beta_p X_p$$

To describe the properties and characteristics of the multiple linear regression (MLR) model, we introduce the `Birthweight` example.

Introduction to the `Birthweight` Example

The goal of the birthweight study is to **predict** the birthweight (*response/ outcome*) of a newborn baby given a set of *predictors/features*, specifically for babies born prematurely.

The features include baby-related characteristics, such as length, weight, head circumference, gestation period, and parent characteristics, such as mother's/father's height, smoking habits, mother's pre-pregnancy weight, mother's/father's age, etc.



The **predictors** we consider may be:

- quantitative, such as length, weight, or gestation period;
- qualitative/categorical, such as smoking habits (yes/no); or
- transformations of quantitative inputs. Examples include power transformations (e.g., X_3^3), continuous functions of variables (e.g., $\log X_5$), polynomial expansions (e.g., $\sum_{k=1}^{n} c_k X_7^k$), and interactions (e.g., $X_1 \cdot X_2$).

2.1 Multiple Linear Regression (MLR) Model

The model formulation is given by:

Multiple Linear Regression Model

$$Y = \beta_0 + \beta_1 X_1 + ... + \beta_p X_p + \varepsilon$$

where

- β_0 is the intercept
- β_j is the regression coefficient associated with predictor X_j
- ε is the error term The usual assumptions for the error terms are $\varepsilon \sim IID(0, \sigma^2\mathbf{I})$

Given the **training data** $\{x_{i1},\ x_{i2}, \ldots, x_{ip};\ y_i\}_{i=1}^{n}$, we can rewrite the model as

$$y_i = \beta_0 + \beta_1 x_{i1} + ... + \beta_p x_{ip} + \varepsilon_i,\ \ i = 1, \ldots, n$$

where n is the sample size. Using the following matrix representation for the

response and the features:

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \quad = \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix} \quad = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} \quad \mathbf{X} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix}$$

the model equation can be written in as

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}$$

which leads to the concise form

$$\underset{\substack{\uparrow \\ \text{Response}}}{\mathbf{y}_{n\times 1}} = \underset{\substack{\uparrow \\ \text{Design} \\ \text{Matrix}}}{\mathbf{X}_{n\times(p+1)}} \underset{\substack{\uparrow \\ \text{Coefficients}}}{{}_{(p+1)\times 1}} + \underset{\substack{\uparrow \\ \text{Error} \\ \text{Term}}}{{}_{n\times 1}}$$

where n is the sample size, and $p+1$ is number of columns of $\mathbf{X}$ (the p predictors plus the intercept).

2.2 MLR Model Fitting

Given the training data $\{x_{i1},\ x_{i2}, \ldots, x_{ip};\ y_i\}_{i=1}^{n}$, we want to estimate , i.e. express:

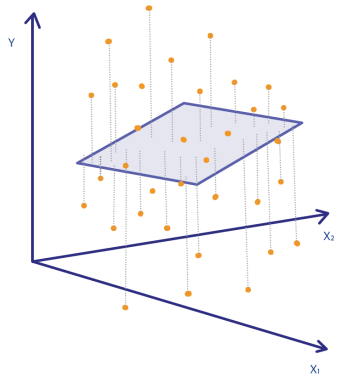
$$\hat{} = \left(\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_p\right)^T$$

as a **function of the data**.

The estimated regression coefficients $\hat{\beta}$ are obtained by minimizing the *Residual Sum of Squares* (RSS):

$$RSS = ||\mathbf{y} - \mathbf{X}\beta||^2 = (\mathbf{y} - \mathbf{X}\beta)^T(\mathbf{y} - \mathbf{X}\beta)$$

The RSS minimizes the (Euclidean) distance of the points from the regression surface:



This approach makes minimal assumptions, since it only requires that the pairs (x_i, y_i) are *random draws from their population*. Specifically, it makes **no assumptions** about the underlying distribution of the response. It simply finds the **best linear fit**, without assuming that the model is valid. In fact, the true underlying model does not even need to be linear for this method to produce estimators. Therefore, it often works well regardless of how the data were obtained. If a linear model provides a good approximation to a non-linear truth, then the regression surface can also be viewed as a measure **lack-of-fit**.

Least-Squares Method

We want to estimate the **vector** of coefficients by minimizing the *sum of squared residuals*:

$$RSS = ||y - \mathbf{X}\beta||^2 = (y - \mathbf{X}\beta)^T(y - \mathbf{X}\beta)$$

We take derivatives with respect to β and set them equal to zero:

$$\frac{\partial RSS}{\partial \beta} = \mathbf{0}_{p\times 1} \Leftrightarrow$$
$$-2\ \mathbf{X}^T_{p\times n}(y - \mathbf{X}\beta)_{n\times 1} = \mathbf{0}_{p\times 1}$$

This leads to the so-called **Normal Equations**

$$\mathbf{X}^T(y - \mathbf{X}\beta) = \mathbf{0}$$

Solving the Normal Equations

$$(\mathbf{X}^T\mathbf{X})\ \beta = \mathbf{X}^T\ y$$

leads to the

Least Squares Estimators in MLR

$$\hat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\, y$$

We assume that **the rank of X is** $p+1$, i.e. no columns of $\mathbf{X}$ are a linear combinations of the other columns. *Since* $\mathbf{X}$ *has full column rank, the inverse of* $(\mathbf{X}^T\mathbf{X})$ *exists.*

Fitted, Predicted Values & Residuals

The **fitted value** of y_i at $x_i = (x_{i1}, x_{i2}, \ldots, x_{ip})$ is computed as:

$$\hat{y_i} = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} + \ldots + \hat{\beta}_p x_{ip}$$

More generally, using matrix formulation, the **fitted values** of $\mathbf{y}$ are given by

$$\begin{aligned}\hat{\mathbf{y}}_{n\times 1} &= \mathbf{X}\hat{\beta} \\ &= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\, y \\ &= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\, y := \mathbf{H}_{n\times n} y_{n\times 1}\end{aligned}$$

where we define the *hat matrix*

The Hat Matrix

$$\mathbf{H}_{n\times n} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$$

It is called the **hat matrix** because it produces the "*y-hat*" values: $\hat{y} = \mathbf{H}t$.

The **predicted value** of y at a new point $x_i^* = (x_{i1}^*, x_{i2}^*, \ldots, x_{ip}^*)$ is

$$\hat{\mathbf{y}}^* = \hat{\beta}_0 + \hat{\beta}_1 x_{i1}^* + \hat{\beta}_2 x_{i2}^* + \ldots + \hat{\beta}_p x_{ip}^*$$

In matrix form, the **predicted values** are

$$\hat{\mathbf{y}}^* = \mathbf{X}^*\hat{\beta}$$

> Note: The key difference between the fitted and predicted values is that the vector $\mathbf{x}$ belongs to the training data (it has already been observed), whereas $\mathbf{x}^*$ is an **unobserved** feature vector that is **independent** of the training data.

The **residual** at observation i is

$$\begin{aligned}r_i &= y_i - \hat{y}_i \\ &= y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \hat{\beta}_2 x_{i2} - \ldots - \hat{\beta}_p x_{ip}\end{aligned}$$

In matrix form the residual vector is

$$\begin{aligned}\mathbf{r}_{n\times 1} &= \mathbf{y} - \hat{\mathbf{y}} \\ &= \mathbf{y} - \mathbf{X}\hat{\beta} = \mathbf{y} - \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\, y \\ &= \mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\mathbf{y}\end{aligned}$$

The residuals **r** are used to estimate the **error variance**:

$$\hat{\sigma}^2 = \frac{1}{n-p-1}\sum_i r_i^2 = \frac{RSS}{n-p-1}$$

The denominator $n-p-1$ is the *degrees of freedom of the residuals* .

Properties of the Residuals

The least-squares estimator $\hat{\beta}$ satisfies the **normal equations**

$$\mathbf{X}^T(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{X}^T(\mathbf{y} - \mathbf{X}\hat{\beta}) = \mathbf{0}$$

From this we obtain the following properties of the residual vector **r**:

- The residual vector is orthogonal to every column of **X**:

$$\begin{aligned}\mathbf{X}^T\,\mathbf{r} &= \mathbf{X}^T(\mathbf{y} - \mathbf{X}\hat{\beta}) \\ &= \mathbf{X}^T\mathbf{y} - \mathbf{X}^T\mathbf{X}\hat{\beta} \\ &= \mathbf{X}^T\mathbf{y} - (\mathbf{X}^T\mathbf{X})(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\,\mathbf{y} = \mathbf{0}\end{aligned}$$

- The residual vector is also orthogonal to the vector of fitted values:

$$\hat{\mathbf{y}}^T\,\mathbf{r} = \hat{\beta}^T\mathbf{X}^T\,\mathbf{r} = \mathbf{0}$$

  Thus, the residual vector **r** is **orthogonal to each column of X and to $\hat{\mathbf{y}}$**.

When the model is used for prediction at a new point $\mathbf{x}^*$, two different interval estimates are commonly reported: (i) a confidence interval for the mean response $E(Y \mid \mathbf{x}^*)$, and (ii) a prediction interval for a new individual observation Y^*. The prediction interval is wider because it must also account for the irreducible error variance σ^2.

2.3 Least-Squares & Normal Equations

In this section, we focus on the system of linear equations that we solve to obtain the least-squares estimators for β. Recall, that we want to find a vector $\widehat{\beta}$ that minimizes

$$\min_{\beta} ||\mathbf{y} - \mathbf{X}\beta||^2$$

Any vector that provides a minimum value of this expression is called a least-squares solution.

The set of all least squares solutions is precisely the **set of solutions** to the normal equations

$$(\mathbf{X}^T\mathbf{X})\beta = \mathbf{X}^T\mathbf{y}$$

There is a *unique* solution **if and only if** $rank(\mathbf{X}) = p+1$ in which case $(\mathbf{X}^T\mathbf{X})$ is invertible.

System of Linear Equations

Linear Algebra Review Let us review a few facts from linear algebra related to solutions of a system of linear equations. For simplicity of notation, we use the *generic* system of equations:

$$\mathbf{A}z = c$$

where

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1k} \\ a_{21} & a_{22} & \cdots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mk} \end{pmatrix}, \quad z = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{pmatrix}, \quad c = \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_m \end{pmatrix}$$

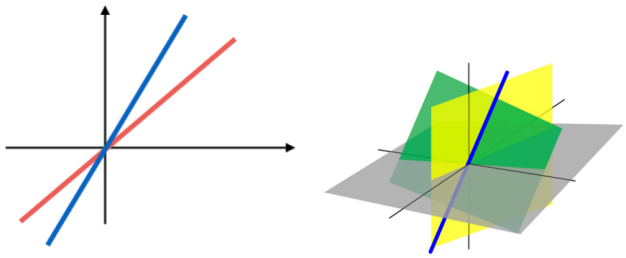
Denote by $\mathbf{A}_i = (a_{1i}, a_{2i}, \dots, a_{mi})^T$ the vector representing the ith *column* of $\mathbf{A}$. Then, $\mathcal{C}(\mathbf{A})$ is the space generated by the columns of $\mathbf{A}$, i.e. the span of $\{\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k\}$.

Definition

The **span** of a collection of vectors $(\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k)$ is the set of all linear combinations of these vectors:

$$\begin{aligned} &span(\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k) \\ &= \left\{ d_1\mathbf{A}_1 + d_2\mathbf{A}_2 + \dots + d_k\mathbf{A}_k, \text{ for any constants } d_1, \dots, d_k \in \mathbb{R} \right\} \end{aligned}$$

The column space of $\mathbf{A}$, $\mathcal{C}(\mathbf{A})$, is a **subspace** in $\mathbb{R}^m$. A linear subspace of $\mathbb{R}^m$ can be thought of as a "flat" surface within $\mathbb{R}^m$; it is a collection of vectors that is closed under linear combinations. Illustrations of subspaces of $\mathbb{R}^n$ are shown below:



Solving the System of Equations $\mathbf{A}z = c$

For the system $\mathbf{A}z = c$ to have a solution, c must lie in the column space of $\mathbf{A}$, i.e. $c \in \mathcal{C}(\mathbf{A})$. This follows directly from the definition of matrix multiplication:

$$\mathbf{A}z = c \Leftrightarrow c = z_1\mathbf{A}_1 + ... + z_k\mathbf{A}_k$$

When $c \notin \mathcal{C}(\mathbf{A})$, we instead seek the vector $\hat{c} \in \mathcal{C}(\mathbf{A})$ that is closest to c. In that case, the equation $\mathbf{A}z = \hat{c}$ has a solution, and $\hat{c}$ comes as close as possible to the original vector c. To obtain $\hat{c}$ we **project** c **orthogonally** onto $\mathcal{C}(\mathbf{A})$ by multiplying both sides by $\mathbf{A}^T$:

$$\mathbf{A}^T\mathbf{A}z = \mathbf{A}^Tc$$

* These are the normal equations.

In the sketch to the right, $\mathcal{C}(\mathbf{A})$, the space spanned by the columns of $\mathbf{A}$, is a *flat* surface, and c is a point lying off that flat surface. The **shortest distance** from the point c to the plane $\mathcal{C}(\mathbf{A})$ is the line segment *orthogonal* to the plane.

The *normal equations* locate the point in $\mathcal{C}(\mathbf{A})$ that is closest to c by means of an orthogonal projection.

Geometric Representation of LS

Translating the preceding discussion into the linear-regression setting:

- $\mathcal{C}(\mathbf{X})$ is the subspace *spanned* by the columns of the design matrix, i.e. by the predictors X_1, X_2, ..., X_p. It is a *flat* subspace of $\mathbb{R}^n$.

- The response $\mathbf{y}$ is a vector lying **off** this subspace, and the fitted value $\hat{\mathbf{y}}$ is the orthogonal projection of $\mathbf{y}$ onto $\mathcal{C}(\mathbf{X})$.
- The residual vector $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$ is **orthogonal** both to $\hat{\mathbf{y}}$ and to the entire subspace $\mathcal{C}(\mathbf{X})$.

In other words, the least-squares method decomposes the data vector $\mathbf{y}$ into two orthogonal components:

$$\mathbf{y}_{n\times 1} = \hat{\mathbf{y}}_{n\times 1} + \mathbf{r}_{n\times 1}$$

2.4 Goodness-Of-Fit: R-Square

A measure of how well the model fits the data is the R-**square** (also called **coefficient of determination** or the *percentage of variance explained*). It is defined as

$$\begin{aligned} R^2 = \frac{\sum_i (\hat{y}_i - \bar{y})^2}{\sum_i (y_i - \bar{y})^2} &= \frac{\text{distance of model from grand mean}}{\text{distance of observations from grand mean}} \\ &= \frac{||\hat{\mathbf{y}} - \bar{\mathbf{y}}||^2}{||\mathbf{y} - \bar{\mathbf{y}}||^2} \\ &= \frac{||\mathbf{y} - \bar{\mathbf{y}}||^2 - ||\hat{\mathbf{y}} - \mathbf{y}||^2}{||\mathbf{y} - \bar{\mathbf{y}}||^2} \\ &= 1 - \frac{||\hat{\mathbf{y}} - \mathbf{y}||^2}{||\mathbf{y} - \bar{\mathbf{y}}||^2} := 1 - \frac{RSS}{TSS} \end{aligned}$$

The second-to-last equality follows from the orthogonal decomposition

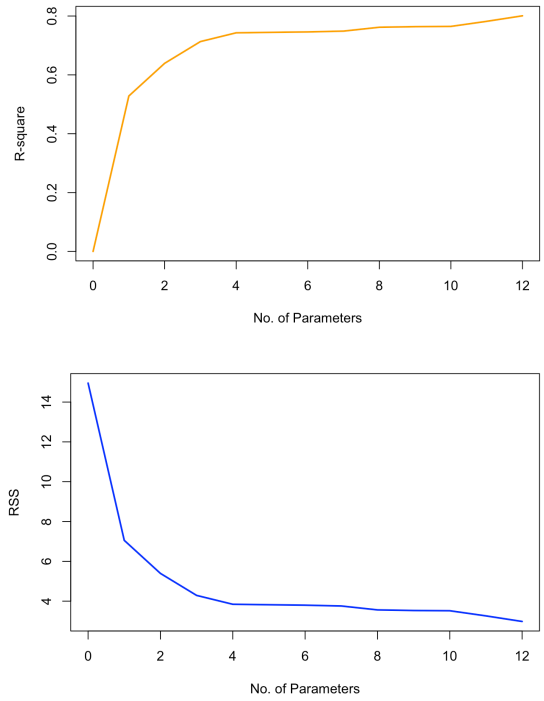
$$\underbrace{||\mathbf{y} - \bar{\mathbf{y}}||^2}_{\substack{\text{total} \\ \text{variation}}} = \underbrace{||\hat{\mathbf{y}} - \bar{\mathbf{y}}||^2}_{\substack{\text{model variation} \\ \text{from mean}}} + \underbrace{||\hat{\mathbf{y}} - \mathbf{y}||^2}_{\text{error}}$$

Properties of R^2

- $0 \leq R^2 \leq 1$
- R^2 is invariant of any location or scale change of Y or of the predictors X.
- R^2 alone does not tell us much about the quality of the LS fit.
- A small R^2 does not necessarily imply that the LS model is poor.

- Adding a new predictor, even if it is randomly generated and completely unrelated to Y, decreases the RSS and therefore increases R^2.

These relationships are illustrated in the two plots below:



2.5 Linear Transformations on X

Suppose we have a linear regression model of Y on the predictors X. If we **scale** or **shift** a predictor, the fitted values $\hat{\mathbf{y}}$ and R^2 remain unchanged, but the LS estimators $\hat{\beta}$ change.

Some Examples

- Scaling a predictor, i.e. $\tilde{x}_{i2} = 2x_{i2}$ or $\tilde{x}_{i3} = x_{i3}/4$, does not affect $\hat{\mathbf{y}}$ or R^2.
- In the `birthweight` example, the length of a baby is recorded in cm. Converting the length column to inches by the transformation $\tilde{X}_1 = X_1/2.54$ leaves the predicted values unchanged.

These statements remain true, if we apply any linear transformation of the p predictors **as long as the transformation does not change the rank of X**.

2.6 Rank deficiency

The design matrix $\mathbf{X}$ is an $n \times (p+1)$ matrix. If this matrix is **not of full rank** (i.e., its columns are not linearly independent), the matrix $\mathbf{X}^T\mathbf{X}$ cannot be inverted. This means that there is redundancy among the columns of $\mathbf{X}$: at least one column can be written as a linear combination of other columns. Consequently, $\mathbf{X^T X}$ is *singular* and the LS solution is not unique (an *identifiability problem*).

Nevertheless, the column space $\mathcal{C}(\mathbf{X})$ remains well-defined, so the fitted vector $\hat{y}$ is still well-defined and can be computed. `R` and `Python` handle this situation, and the resulting model can still be used for prediction.

2.7 Hypothesis Testing in MLR

When fitting a regression model, we often want to understand which predictors are statistically significant, or which of two models (a larger one or a smaller one) is more appropriate –either for prediction or for estimation.

Testing for the statistical significance of one or more predictors can be formulated either as a test that the corresponding coefficient(s) equal zero, or as a model-comparison test. In this course we summarize all such testing questions with the following hypothesis test:

$$\begin{cases} H_0: \text{ Reduced Model with } p_0 \text{ coefficients} \\ H_\alpha: \text{ Full (larger) Model with } p_\alpha \text{ coefficients} \end{cases}$$

Obviously, $p_\alpha > p_0$, since the reduced model is a special case of the full model. Consequently we expect the reduced model to have a larger residual sum of squares: $RSS_0 \geq RSS_\alpha$. The main tool for this comparison is the **partial** F **test**:

Partial F **test**

Hypothesis Test:

$$\begin{cases} H_0: \text{ Reduced Model with } p_0 \text{ coefficients} \\ H_\alpha: \text{ Full/ Larger Model with } p_\alpha \text{ coefficients} \end{cases}$$

Test Statistic:

$$F = \frac{(RSS_0 - RSS_\alpha)/(p_\alpha - p_0)}{RSS_\alpha/(n - p_\alpha)} \sim F_{p_\alpha - p_0, n - p_\alpha}$$

Decision Rule: Reject H_0, if the F test statistic is large (or larger than $F_{p_\alpha - p_0, n-p_\alpha}$), i.e. the variation missed by the reduced model, when being compared with the error variance, is significantly large.

- The **numerator** in the partial F test measures the extra variation in the data not explained by the reduced model, but explained by the full model.
- The **denomimator** is the variation in the data not explained by the full model (i.e., not explained by either model), which is used to estimate the error variance.

Birthweight Example (cont'd)

We wish to test whether the group of predictors that describe the father's characteristics (i.e. variables X_9 – X_{12}) is jointly significant. The hypotheses are

$$\begin{cases} H_0 : Y \sim 1 + \beta_1 X_1 + ... + \beta_8 X_8 \\ H_\alpha : Y \sim 1 + \beta_1 X_1 + ... + \beta_8 X_8 + ... + \beta_{12} X_{12} \end{cases}$$

Here $p_0 = 8 + 1$ (8 predictors plus intercept) and $p_\alpha = 12 + 1$ is the number of β's in the full model. Extracting RSS_0 and RSS_α in `R/Python` and computing the partial F test statistic yields $F = 0.7225$. Because this value is smaller than the critical value $ F_{4,, n-13} 2.7$ we do not reject H_0 and prefer the reduced model.

Partial F Test: Special Cases

- **Test for a Single Predictor β_j**

$$\begin{cases} H_0 : Y \sim 1 + \beta_1 X_1 + ... + \beta_{j-1} X_{j-1} + \qquad \beta_{j+1} X_{j+1} + ... + \beta_p X_p \\ H_\alpha : Y \sim 1 + \beta_1 X_1 + ... + \beta_{j-1} X_{j-1} + \beta_j X_j + \beta_{j+1} X_{j+1} + ... + \beta_p X_p \end{cases}$$

The partial F test for a single coefficient β_j is exactly equivalent to the usual t-test$

$$t_j = \frac{\hat{\beta}_j}{\widehat{\text{se}}(\hat{\beta}_j)} \sim t_{n-p-1}$$

under $H_0 : \beta_j = 0$. In practice most software reports the t-statistic and its p-value directly.

- **Test for all Predictors**

$$\begin{cases} H_0 : Y \sim 1 \qquad \text{[intercept-only model]} \\ H_\alpha : Y \sim 1 + \beta_1 X_1 + ... + \beta_{j-1} X_{j-1} + \beta_j X_j + \beta_{j+1} X_{j+1} + ... + \beta_p X_p \end{cases}$$

This is the **overall** F test, which can be viewed as a *goodness-of-fit test* for the regression model.

2.8 Categorical Variables in MLR

As we discussed, some of the predictors may be *categorical*. Consider a categorical predictor with k levels. When it is included in a linear regression model it is represented it is represented by binary indicator variables. Two common coding schemes are used:

One-hot encoding creates $k $ binary indicators, one for each level:

$$D_i = \begin{cases} 1 & \text{if the observation belongs to level } i, \\ 0 & \text{otherwise.} \end{cases}$$

Dummy coding (the default in classical linear regression) creates only $ k-1 $ binary indicators and absorbs one level into the intercept. The indicators take the form

$$D_i = \begin{cases} 0 & \text{if the observation is not at level } i, \\ 1 & \text{if the observation is at level } i, \end{cases}$$

where the omitted level is called the reference level.

An Example with Categorical Variables

Consider a categorical predictor with three levels (a, b, c).

One-hot encoding produces $k = 3$ columns:

$$\begin{pmatrix} a \\ a \\ b \\ b \\ b \\ c \\ c \end{pmatrix} \quad \Longrightarrow \quad \begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

Dummy coding (with level c as the reference) produces $k-1=2$ columns:

$$\begin{pmatrix} a \\ a \\ b \\ b \\ b \\ c \\ c \end{pmatrix} \quad \Longrightarrow \quad \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

In the dummy-coding matrix, Column 1 is the indicator for level a and Column 2 is the indicator for level b. Level c is absorbed into the intercept.

> In general any level may be chosen as the reference level; the choice is usually driven by subject-matter context rather than by statistical considerations.

Dummy coding vs. One-hot encoding

- Dummy coding uses $k-1$ indicators and keeps the design matrix full rank when an intercept is present. This is the standard approach in classical linear regression.
- One-hot encoding uses k indicators. When an intercept is also included, the k columns sum to the constant vector $\mathbf{1}$, creating perfect multicollinearity; $\mathbf{X}^T\mathbf{X}$ becomes singular and the ordinary least-squares estimator is no longer unique.
- Fitted and predicted values are identical under both codings. *Only the interpretation of the individual coefficients changes.* In practice, any level may be chosen as the reference; the choice is usually guided by subject-matter context rather than by statistical considerations.

2.9 Collinearity

In practice we often encounter data sets in which many of the predictors are highly correlated with one another. In such cases the fitted values and the sampling variances of the regression coefficients can depend heavily on which particular predictors are included in the model. The resulting estimators become sensitive and unreliable.

Possible *symptoms* of collinearity include :

(i) high pairwise (sample) correlation among the predictors,

(ii) a relatively large R^2,

(iii) a statistically significant overall F test even though non of the individual predictors is statistically significant.

Specific diagnostic tests and metrics exist for quantifying the severity of collinearity, but they lie beyond the scope of this course.

The simplest remedy is to remove one or more predictors from each highly correlated group. More advanced approaches include principal-component analysis (PCA) and regularization methods based on penalized least squares.

2.10 Model Diagnostics

We made several assumptions when defining the linear regression model

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon, \quad \text{where} \quad \varepsilon \sim IID(0, \sigma^2)$$

These assumptions are:

- *Linearity*: the conditional mean of Y is a linear function of the predictors.
- *Constant Variance*(homoscedasticity): all responses share a common, constant variance σ^2.
- *Uncorrelated errors*: the errors ε_i and ε_j (and therefore Y_i and Y_j) are uncorrelated for $i \neq j$.
- *(Normality)*: this is not required for the derivation of the least-squares estimators themselves. It becomes necessary, however, when we want exact finite-sample distributional results for the estimators, fitted values, and residuals, and it is especially important for hypothesis testing.

If the goal is to interpret the effects of the predictors on the response (rather than pure prediction), all of the above assumptions should be checked with diagnostic plots and tests, and appropriate remedial measures should be taken when they are violated.

Because the primary focus of this course is prediction, we refer students to a dedicated regression course or textbook for a thorough treatment of diagnostics and remedial measures in multiple linear regression.

Outliers

Outliers can be a concern in both large and small data sets. When the sample size is modest, it is advisable to test formally for outliers—for example, with tests based on the leave-one-out prediction error. In very large data sets such

tests lose power and are less useful; one should instead adjust for multiple comparisons or adopt alternative strategies.

If outliers appear to be problematic, useful practical steps include:

(i) examining the range of each variable,

(ii) applying a log, square-root, or other suitable transformation to right-skewed predictors or to the response Y, and

(iii) using winsorization to limit the influence of extreme values.

2.11 The `Birthweight` Example

The `birthweight.csv` data set contains observations on 42 babies. The variables are:

- `ID`: Unique Identification number of a baby
- `Length`: Length of the baby at time of birth in cm (X_1)
- `Birthweight`: Weight of the baby at time of birth in kg (Y)
- `Headcirc`: Head circumference of the baby at time of birth in cm (X_2)
- `Gestation`: Completed weeks of gestation (X_3)
- `smoker`: Mother is/is not a smoker {0: No, 1: Yes} (X_4)
- `mage`: Mother's age at time of birth (X_5)
- `mnocig`: Mother's number of cigarettes smoked per day (X_6)
- `mheight`: Mother's height in cm (X_7)
- `mppwt`: Mother's pre-pregnancy weight (X_8)
- `fage`: Father's age at time of birth (X_9)
- `fedyrs`: Father's years of education (X_{10})
- `fnocig`: Father's number of cigarettes smoked per day (X_{11})
- `fheight`: Father's height (X_{12})

You can download the data file here: Birthweight.csv .

The analysis of this data set is available in R and Python.

The data analysis example also includes a highly correlated data set (`seatpos`) from Faraway's book on "Linear Models in R". You can download this data here: seatpos.csv

Chapter 3

Variable Selection & Regularization

Consider the multiple linear regression model with p predictors **plus** an intercept:

$$Y \sim 1 + X_1 + X_2 + ... + X_p$$

In many applications the number of explanatory variables p is large; in some cases we even have $p \gg n$. This does not mean that every variable is relevant to the response Y. In fact, *only a small fraction* of the p variables is typically believed to be relevant.

Our **goal** in this chapter is to develop methods that efficiently identify the predictors that are useful for estimating or predicting the response. Because the least-squares estimator $\widehat{\beta}$ is unbiased, the coefficients of irrelevant variables tend to be close to zero. Nevertheless, if our primary objective is good prediction, we must still ask whether it is beneficial to remove those unnecessary variables from the model.

To understand the consequences of including superfluous predictors, we first examine the **training** and **testing** errors more carefully.

3.1 Training vs. Testing Errors

Consider that we *split our data in two parts*:

- **Training data** $\{\mathbf{x}_i, y_i\}_{i=1}^n$: used to fit the model

- **Testing data** $\{\mathbf{x}_i, y_i^*\}_{i=1}^n$: an **independent** data set collected at the same locations $\mathbf{x}_i$

Remark: In practice we are given a single data set that we randomly partition into a training set and a testing set (typically allocating roughly 80 % of the observations to training).

We assume that both *testing* and *training* observations come from the same population (or are observed at the same design points $\mathbf{x}_i$). Statistically we may therefore write

$$\mathbf{y}_{n\times 1},\ \mathbf{y}^*_{n\times 1} \sim^{iid} N_n(\ , \sigma^2\mathbf{I}_n) \text{ and } \quad = \mathbf{X}$$

or equivalently,

$$\mathbf{y} = \mathbf{X}\ +\ ,\quad \sim^{iid} \mathcal{N}_n(\mathbf{0}, \sigma^2\mathbf{I}_n) \quad \text{and} \quad \mathbf{y}^* = \mathbf{X}\ +\ ^*,\ ^* \sim^{iid} \mathcal{N}_n(\mathbf{0}, \sigma^2\mathbf{I}_n)$$

$$\text{with} \quad ,\ ^* \ \text{independent.}$$

Having these models in mind, we compute the **MSE for train and testing data**:

For simplicity we suppress the intercept in the calculations below (so the model dimension is p rather than $p+1$). The expected training and testing mean squared errors are then:

$$\begin{aligned}
\mathbb{E}(\text{Train Error}) &= \mathbb{E}\left\|\mathbf{y} - \hat{\mathbf{y}}\right\|^2 = \mathbb{E}\left\|(\mathbf{I}-\mathbf{H})\mathbf{y}\right\|^2 \\
&= tr\Big((\mathbf{I}-\mathbf{H})Cov(\mathbf{y})(\mathbf{I}-\mathbf{H})^T\Big) \\
&= \sigma^2 tr\Big((\mathbf{I}-\mathbf{H})\Big) = (n-p)\sigma^2 \\
&= n\sigma^2 - p\sigma^2
\end{aligned}$$

$$
\begin{aligned}
\mathbb{E}(\text{Test Error}) &= \mathbb{E}\left\|\mathbf{y}^* - \mathbf{X}\hat{}\right\|^2 \\
&= \mathbb{E}\left\|(\mathbf{y}^* - \mathbf{X}\ ) + (\mathbf{X}\ - \mathbf{X}\hat{}\,)\right\|^2 \\
&= \mathbb{E}\left\|\mathbf{y}^* - \ \right\|^2 + \mathbb{E}\left\|\mathbf{X}\ - \mathbf{X}\hat{}\right\|^2 \\
&= \mathbb{E}||\ ^*||^2 + tr\Big(\mathbf{X}Cov(\hat{})\mathbf{X}^T\Big) \\
&= n\sigma^2 + \sigma^2 tr\mathbf{H} \\
&= n\sigma^2 + p\sigma^2
\end{aligned}
$$

From these expressions we see that

- the *training error decreases with* p .
- the *testing error increases with* p .

Consequently, if the sole objective is pure prediction, simply adding more columns to $\mathbf{X}$ is not automatically beneficial. Does this imply that the intercept-only model ($p = 0$)—the model with the smallest expected test error under the calculation above—is optimal? **No.**

The preceding analysis rests on two important **assumptions**:

1. The conditional mean $E(\mathbf{Y}|\mathbf{X})$ lies in the column space of $\mathbf{X}$; that is, there exists a coefficient vector β such that $\mathbb{E}(\mathbf{Y} \mid \mathbf{X}) = \mathbf{X}\beta$.
2. The design matrix $\mathbf{X}$ contains *all* available predictors. When we fit a linear model that uses only a **subset** of the columns of $\mathbf{X}$, an additional **bias** term appears.

More generally we may face a model of the form

$$Y = f(X) + \varepsilon$$

in which f is not linear.

Approximating f by a linear function f^* introduces bias. Even when the true relationship is linear, omitting relevant predictors likewise produces bias.

In the presence of bias the expected errors become:

$$\mathbb{E}(\text{Test Error})^2 = n\sigma^2 + p\sigma^2 + Bias$$
$$\mathbb{E}(\text{Training Error})^2 = n\sigma^2 - p\sigma^2 + Bias$$

where $p\sigma^2$ is the unavoidable error – the ε – and Bias is the model error.

- Larger model (p large) $\rightarrow$ small Bias, large Variance ($p\sigma^2$).
- Smaller model (p small) $\rightarrow$ large Bias, small Variance ($p\sigma^2$).

Reducing the test (prediction) error therefore requires finding a good **trade-off between bias and variance**.

3.2 Subset Selection

The basic idea of *subset selection* is to score every candidate model by an information criterion that balances

$$\text{Training Error} + \text{Complexity Penalty}$$

and then to search for the model with the best score. For linear regression the complexity of a model increases with the number of predictors p.

- **Training Error**: an increasing function of RSS.
- **Complexity Penalty**: an increasing function of p.

> Why not simply use R^2 or RSS? Both R^2 and RSS improve (or at least never worsen) whenever a new variable is added, so they do not penalize the inclusion of irrelevant predictors.

3.2.1 Information Criteria

Akaike Information Criterion & Bayesian Information Criterion

AIC and BIC are defined as

$$AIC = -2 \cdot loglik + 2\ p$$
$$BIC = -2 \cdot loglik + \log(n)\ p$$

where p is the number of predictors in the model under consideration and $\log \text{lik}$ is the maximized log-likelihood. For the normal-error linear model the first term reduces to

$$-2 \cdot loglik = n \log \frac{RSS}{n}$$

(up to an additive constant that does not affect model comparison). Consequently

$$AIC = n \log \frac{RSS}{n} + 2\, p$$
$$BIC = n \log \frac{RSS}{n} + \log(n)\, p$$

Lower values of AIC or BIC are preferred. When n is large, the penalty for an extra predictor is substantially heavier under BIC than under AIC; thus AIC tends to select larger models than BIC.

Adjusted-R^2 for model with p predictors

$$\begin{aligned} R^2_a &= 1 - \frac{RSS/(n-p-1)}{TSS/(n-1)} \\ &= 1 - (1-R^2)\Big(\frac{n-1}{n-p-1}\Big) \\ &= 1 - \frac{\hat{\sigma}^2}{\hat{\sigma}^2_0} \end{aligned}$$

where $\hat{\sigma}^2$ is the residual variance estimate for the current model and $\hat{\sigma}^2_0$ is the residual variance estimate for the intercept-only model.

Higher values of R^2_a are preferred.

Mallow's C_p

$$C_p = \frac{RSS_{\mathbf{p}}}{\hat{\sigma}^2_0} + 2p - n$$

where $\hat{\sigma}^2_0$ is the residual variance estimate obtained from the *full* model. Mallows' C_p behaves similarly to AIC; **lower** values are preferred.

Illustration of Complexity Criteria vs. p

The figure below shows how the adjusted R^2, Mallows' C_p, AIC and BIC change with the number of predictors p for the `student-grades` data set introduced in the previous section.

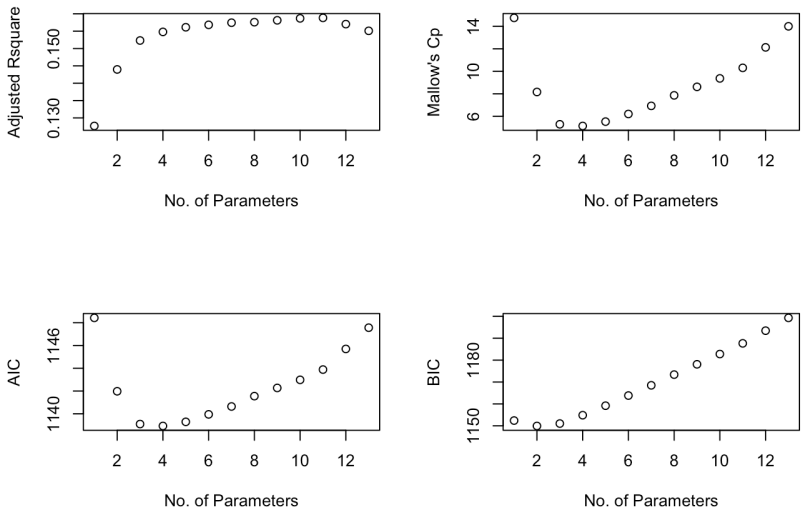


Figure 3.1: Subset selection criteria vs. p.

3.2.2 Search Algorithms

With p available predictors there are 2^p possible models. When p is modest we can evaluate all of them; when p is large we need systematic search strategies.

Level-wise search algorithms: These procedures return a globally optimal solution among all possible models and are practical for up to roughly 40 variables.

The idea is that they identify the m different models (default $m = 8$ in `R`) that have the smallest residual sum of squares, then score those models with the chosen information criterion. The algorithm need not examine every subset: if, for example, $RSS(X_1, X_2) < RSS(X_3, X_4, X_5, X_6)$, then every size-2 or size-3 sub-model of X_3, X_4, X_5, X_6 can be safely skipped ("leaped over").

Greedy algorithms: These algorithms add or remove predictors according to an information criterion (AIC, BIC, …). Three common variants are:

- *Backward elimination*: start with the full model and successively delete the predictor whose removal most improves the criterion, until no further improvement is possible;
- *Forward selection*: start with the null model and successively add the predictor that most improves the criterion;
- *Stepwise selection*: at each step consider both adding and deleting a single predictor.

Greedy methods are computationally efficient but return only a *locally optimal* model; in practice this is usually satisfactory.

Some practical considerations

When $p \gg n$ it is infeasible to begin with the full model. A common screening strategy is therefore:

- Rank the p predictors by the absolute value of their marginal correlation with Y.
- Retain only the top K predictors (e.g., $K = n/3$).
- Apply a stepwise procedure that is still allowed to re-introduce previously discarded variables.

3.3 Shrinkage Methods

The variable-selection methods discussed earlier work well when the number of predictors is moderate (for example, level-wise algorithms are practical for $p < 40$). In this section we study two shrinkage methods that can handle larger p by continuously shrinking coefficient estimates toward zero, thereby *balancing the bias–variance trade-off.*

We can re-frame the earlier approaches as a single penalized least-squares problem:

Regression with Penalization

$$\begin{aligned}\hat{\beta} &= \arg\min_{\beta} ||\mathbf{y} - \mathbf{X}\beta||^2 + \underbrace{\lambda \sum_{j=1}^{p} \mathbf{1}_{\beta_j \neq 0}}_{\text{penalty}} \\ &= \arg\min_{\beta} ||\mathbf{y} - \mathbf{X}\beta||^2 + \underbrace{\lambda ||\beta||_0}_{\text{penalty}}\end{aligned}$$

where $||\beta||_0 = \sum_{j=1}^{p} \mathbf{1}_{\beta_j \neq 0}$ simply counts the number of non-zero coefficients. Different choices of the penalty parameter λ recover the information criteria we studied previously.

Replacing the ℓ_0 "norm" by a convex surrogate yields two of the most widely used *penalized regression* methods:

- **Ridge Regression**

$$\hat{\beta} = \arg\min_{\beta} ||\mathbf{y} - \mathbf{X}\beta||^2 + \underbrace{\lambda ||\beta||^2}_{\text{penalty}}, \text{ where } ||\beta||^2 = \sum_{j=1}^{p} \beta_j^2$$

- **Lasso Regression**

$$\hat{\beta} = \arg\min_{\beta} ||\mathbf{y} - \mathbf{X}\beta||^2 + \underbrace{\lambda |\beta|}_{\text{penalty}}, \text{ where } |\beta| = \sum_{j=1}^{p} |\beta_j|$$

Before examining the mathematics and implementation of these methods, we note two important practical considerations.

Implementation Considerations

Neither the ℓ_1 nor the ℓ_2 penalty is invariant to location or scale. Consequently the penalized estimators are sensitive to the units in which the predictors are measured. It is therefore standard practice to **center and scale** each column of the design matrix:

$$\tilde{\mathbf{x}}_j = \frac{\mathbf{x}_j - \bar{\mathbf{x}}_j}{sd_{\mathbf{x}_j}}$$

and to center the response,

$$\tilde{\mathbf{y}}_i = \mathbf{y}_i - \bar{\mathbf{y}}$$

After model fitting the coefficients can be transformed back to the original scale for interpretation and prediction. The intercept is recovered by

$$\hat{\beta}_0 = \bar{y} - \sum_{j=1}^{n} \hat{\beta}_j \bar{\mathbf{x}}_j$$

The R package `glmnet` performs the centering, scaling, and back-transformation automatically.)

In Python, `scikit-learn`'s Ridge, Lasso, and ElasticNet do not automatically center and scale the predictors. You should do this yourself—most commonly by placing a StandardScaler inside a Pipeline. The fitted coefficients are then returned on the scaled scale; you can transform them back to the original scale if needed. There is also a Python port of the original glmnet Fortran code that mirrors the R package's automatic standardization and back-transformation behavior.

3.3.1 Ridge Regression

Ridge regression shrinks the coefficients of a linear model by adding an ℓ_2 penalty. It is especially useful when predictors are highly collinear and ordinary least-squares estimates become unstable. The method was originally introduced by Tikhonov as a way to stabilize the normal equations by adding a non-negative constant to the diagonal of $\mathbf{X}^\top\mathbf{X}$.

Ridge Regression

Solve

$$\text{minimize}_\beta\,(y - X\beta)^T(y - X\beta) + \lambda\sum_j \beta_j^2$$

for a fixed $\lambda \geq 0$.

After the predictors have been standardized and the response centered, the ridge estimator admits the **closed-form** expression

$$\hat{\beta}_{\text{Ridge}} = (X^T X + \underbrace{\lambda I}_{\text{ridge}})^{-1} X^T y$$

When $\lambda = 0$ the estimator reduces to ordinary least squares; as $\lambda \to \infty$ the coefficients shrink to zero. In practice λ is chosen by cross-validation (or generalized cross-validation). A well-known drawback is that the ridge estimator is biased: $\mathbb{E}(\hat{\beta}_{\text{ridge}}) = \beta + \text{bias}$.

To gain intuition, first suppose the columns of $\mathbf{X}$ are orthonormal, so that $\mathbf{X}^\top\mathbf{X} = \mathbf{I}_p$. Then

$$\hat{\beta}_{ridge} = \left(\mathbf{X}^T\mathbf{X} + \lambda I\right)^{-1}\mathbf{X}^T\mathbf{y} = \frac{1}{1+\lambda}\mathbf{X}^T\mathbf{y}$$

and the fitted values satisfy

$$\hat{\mathbf{y}}_{ridge} = \mathbf{X}\hat{\beta}_{ridge} = \frac{1}{1+\lambda}\,\hat{\mathbf{y}}_{LS}$$

When the columns are not orthogonal the same geometric picture can be obtained via the singular-value decomposition (SVD).

Singular Value Decomposition (SVD)

The SVD is a fundamental matrix factorization used throughout applied mathematics, statistics, and machine learning. Principal-component analysis is a special case of the SVD.

SVD of $\mathbf{X}_{n\times p}$

Any matrix $\mathbf{X}_{n\times p}$ admits the factorization

$$\mathbf{X} = \mathbf{U}_{n\times p}\,\mathbf{D}_{p\times p}\,\mathbf{V}^{T}_{p\times p}$$

where

- $\mathbf{U}_{n\times p}$ has orthonormal columns that span $\mathcal{C}(\mathbf{X})$,
- $\mathbf{V}_{p\times p}$ is an orthogonal matrix whose columns span $\mathbb{R}^p$,
- $\mathbf{D}_{p\times p}$ contains the singular values $d_1 \geq ... \geq d_p \geq 0$.

If any $d_j = 0$, then $\mathbf{X}$ is rank-deficient.

This factorization of the matrix $\mathbf{X}$ is called the **singular value decomposition of $\mathbf{X}$**, and the columns of $\mathbf{U}$ and $\mathbf{V}$ are called the left- and right-hand **singular vectors** of $\mathbf{X}$.

For the remainder of the discussion we assume $n > p$ and $\text{rank}(\mathbf{X}) = p$ (so $d_p > 0$).

Useful Properties of the SVD

1. The right singular vectors (columns of $\mathbf{V}$) are orthonormal eigenvectors of $\mathbf{X}^\top\mathbf{X}$.
2. The left singular vectors (columns of $\mathbf{U}$) are orthonormal eigenvectors of $\mathbf{X}\mathbf{X}^\top$.
3. The columns of $\mathbf{V}$ are the *principal-component directions* of the columns of $\mathbf{X}$.
4. The first principal-component direction $\mathbf{v}_1$ maximizes the sample variance of the linear combination $\mathbf{z}_1 = \mathbf{X}\mathbf{v}_1 = \mathbf{u}_1 d_1$ among all unit-norm coefficient vectors.

5. The singular values are the square roots of the eigenvalues of both $\mathbf{X}^\top\mathbf{X}$ and $\mathbf{X}\mathbf{X}^\top$.
6. The largest singular value satisfies

$$d_1 = \max_{\|\mathbf{x}\|=1} \|\mathbf{X}\mathbf{x}\|_2.$$

The principal-component *scores* are the columns of the matrix $\mathbf{S} = \mathbf{U}\mathbf{D}$; they give the coordinates of the observations in the principal-component basis:

$$\mathbf{X} = \mathbf{S}\mathbf{V}^\top.$$

SVD of Fitted Values: LS vs. Ridge

Using the SVD we can write the ordinary-least-squares fitted values as

$$\begin{aligned}
\hat{\mathbf{y}}_{LS} &= \mathbf{X}\hat{\beta}_{LS} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \\
&= \mathbf{U}\ \mathbf{D}\ \mathbf{V}^T(\mathbf{V}\ \mathbf{D}^2\ \mathbf{V}^T)^{-1}\mathbf{V}\ \mathbf{D}\ \mathbf{U}^T\mathbf{y} \\
&= \mathbf{U}\ \mathbf{D}\ \mathbf{V}^T(\mathbf{V}^T)^{-1}\ \mathbf{D}^{-2}\ \mathbf{V}^{-1}\mathbf{V}\ \mathbf{D}\ \mathbf{U}^T\mathbf{y} \\
&= \mathbf{U}\ \mathbf{D}\ \mathbf{D}^{-2}\mathbf{D}\ \mathbf{U}^T\mathbf{y} \\
&= \mathbf{U}\mathbf{U}^T\mathbf{y} \\
&= \sum_{j=1}^{p}(\mathbf{u}_j^T\mathbf{y})\mathbf{u}_j
\end{aligned}$$

(The algebraic steps rely on $\mathbf{X}^\top\mathbf{X} = \mathbf{V}\mathbf{D}^2\mathbf{V}^\top$ and the orthonormality of $\mathbf{U}$ and $\mathbf{V}$.)

For ridge regression the same calculation yields

$$\begin{aligned}
\hat{\mathbf{y}}_{ridge} &= \mathbf{X}\hat{\beta}_{ridge} \\
&= \mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y} \\
&= \mathbf{U}\ \mathbf{D}\ \mathbf{V}^T(\mathbf{V}\ \mathbf{D}^2\ \mathbf{V}^T + \lambda\mathbf{I})^{-1}\mathbf{V}\ \mathbf{D}\ \mathbf{U}^T\mathbf{y} \\
&= \mathbf{U}\ \mathbf{D}\ \mathbf{V}^T(\mathbf{V}\ \mathbf{D}^2\ \mathbf{V}^T + \lambda\mathbf{V}\mathbf{V}^T)^{-1}\mathbf{V}\ \mathbf{D}\ \mathbf{U}^T\mathbf{y} \\
&= \mathbf{U}\ \mathbf{D}\ \mathbf{V}^T(\mathbf{V}\ (\mathbf{D}^2 + \lambda\mathbf{I})\ \mathbf{V}^T)^{-1}\mathbf{V}\ \mathbf{D}\ \mathbf{U}^T\mathbf{y} \\
&= \mathbf{U}\ \mathbf{D}\ \mathbf{V}^T(\mathbf{V}^T)^{-1}\ (\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\ \mathbf{V}^{-1}\mathbf{V}\ \mathbf{D}\ \mathbf{U}^T\mathbf{y} \\
&= \mathbf{U}\mathbf{D}\ (\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\ \mathbf{D}\mathbf{U}^T\mathbf{y}
\end{aligned}$$

Thus ridge regression projects $\mathbf{y}$ onto the principal-component basis and then multiplies the j-th coordinate by the shrinkage factor $d_j^2/(d_j^2+\lambda)$. Components associated with small singular values (low-variance directions) are shrunk more aggressively.

In this last expression note that

- $\mathbf{D}(\mathbf{D}^2+\lambda\mathbf{I})^{-1}$ is a diagonal matrix whose j-th diagonal entry is $\dfrac{d_j^2}{d_j^2+\lambda}$,
- the vector $\mathbf{U}^\top\mathbf{y}$ contains the coordinates of $\mathbf{y}$ in the orthonormal basis spanned by the columns of $\mathbf{U}$.

Consequently the ridge fitted values simplify to

$$\hat{\mathbf{y}}_{\text{ridge}} = \sum_{j=1}^{p} \mathbf{u}_j \cdot \frac{d_j^2}{d_j^2+\lambda} \cdot (\mathbf{u}_j^\top \mathbf{y}).$$

In other words, ridge regression first projects $\mathbf{y}$ onto the principal-component basis and then multiplies each coordinate by the shrinkage factor $\dfrac{d_j^2}{d_j^2+\lambda}$. Equivalently, the ridge estimator shrinks the ordinary-least-squares coefficients by these same factors: the smaller the singular value d_j (i.e., the lower the sample variance of the corresponding principal component), the stronger the shrinkage.

Complexity of Ridge Regression

To quantify the complexity of a model, heuristically we need to understand the number of coefficients that need to be estimated. Ordinary least squares with p predictors has p degrees of freedom. Because ridge regression shrinks coefficients, its effective degrees of freedom lie strictly between 0 and p. If λ is extremely large, there will be no effective covariates left in the model, meaning that they should all be close to zero. On the other hand, if λ is 0, then we go back to linear regression with p covariates and p degrees of freedom.

A convenient definition of **degrees of freedom** is

$$df = \sum_{i=1}^{n} Cor(y_i,\ \hat{y}_i)$$

- For **ordinary least squares** $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ with $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$, so

$$df = \sum_{i=1}^{n} Cor(y_i,\ \hat{y}_i) = \sum_{i=1}^{n} H_{ii} = tr(\mathbf{H}) = p$$

- For **ridge regression** $\hat{\mathbf{y}} = \mathbf{S}_\lambda \mathbf{y}$ where

$$\mathbf{S}_\lambda = \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top.$$

Hence

$$\begin{aligned} df(\lambda) &= \sum_{i=1}^{n} Cor(y_i,\ \hat{y}_i) = \sum_{i=1}^{n} [S_\lambda]_{ii} \\ &= tr(\mathbf{S}_\lambda) \\ &= tr(\mathbf{X}(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T) \\ &= tr\left(\sum_{j=1}^{p} \frac{d_j^2}{d_j^2 + \lambda} \mathbf{u}_i \mathbf{u}_i^T \right) = \sum_{j=1}^{p} \frac{d_j^2}{d_j^2 + \lambda} \end{aligned}$$

The effective degrees of freedom decrease monotonically with λ and equal p when $\lambda = 0$. This expression is often used to choose a grid of λ values that correspond to integer degrees of freedom $k = 1, \ldots, p$.

3.3.2 Lasso Regression

Lasso regression replaces the ℓ_2 penalty by an ℓ_1 penalty.

Lasso Regression

Solve

$$\text{minimize } (y - X\beta)^T (y - X\beta) + \lambda \sum_j |\beta_j|$$

for a fixed $\lambda \geq 0$.

In two-dimensions the lasso constraint defines a square, while in higher dimensions it defines a polytope.

Lasso is useful when the response can be explained by *few* predictors with zero effect on the remaining predictors (Lasso is similar to a variable selection method). When $\beta_j = 0$ the corresponding predictor is eliminated which is not the case for ridge regression. Therefore, we use lasso when the effect of predictors is **sparse**. This means that only few predictors will have an effect on the response (e.g. gene expression data) or when number of predictors is large ($p > n$).

Unlike ridge regression, the lasso estimator *does not admit a closed-form expression in general*. When the columns of $\mathbf{X}$ are orthonormal, however, the problem separates coordinate-wise and the solution is given by soft-thresholding.

The lasso solution is defined as

$$\hat{\beta}_{lasso} = \arg\min_{\beta \in \mathbb{R}^p} \Big((y - X\beta)^\top (y - X\beta) + \lambda \sum_j |\beta_j| \Big)$$

and does not have a closed form expression.

If we assume that $\mathbf{X}^T\mathbf{X} = \mathbf{I}_p$, then

$$\begin{aligned} ||\mathbf{y} - \mathbf{X}\beta||^2 &= ||\mathbf{y} - \mathbf{X}\hat{\beta}_{LS} + \mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta||^2 \\ &= ||\mathbf{y} - \mathbf{X}\hat{\beta}_{LS}||^2 + ||\mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta||^2 \end{aligned}$$

where

$$2(\mathbf{y} - \mathbf{X}\hat{\beta}_{LS})^T (\mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta) = 2\, r^T\, (\mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta) = 0$$

since the second term is a linear combination of columns of $\mathbf{X}$ no matter what value β takes, and thus is in $\mathcal{C}(\mathbf{X})$, therefore orthogonal to the residual vector $\mathbf{r}$.

Obtaining the Lasso Solution

Under the orthonormality assumption $\mathbf{X}^\top\mathbf{X} = \mathbf{I}_p$ the lasso criterion reduces to a sum of p independent one-dimensional problems of the form

$$\arg\min_x (x - a)^2 + \lambda|x|,\ \lambda > 0$$

The lasso solution can be expressed as:

$$\begin{aligned} \hat{\beta}_{lasso} &= \arg\min_{\beta \in \mathbb{R}^p} \Big(||\mathbf{y} - \mathbf{X}\beta||^2 + \lambda|\beta| \Big) \\ &= \arg\min_{\beta \in \mathbb{R}^p} \Big(||\mathbf{X}\hat{\beta}_{LS} - \mathbf{X}\beta||^2 + \lambda|\beta| \Big) \\ &= \arg\min_{\beta \in \mathbb{R}^p} \Big((\hat{\beta}_{LS} - \beta)^T \mathbf{X}^T\mathbf{X} (\hat{\beta}_{LS} - \beta) + \lambda|\beta| \Big) \\ &= \arg\min_{\beta \in \mathbb{R}^p} \Big((\hat{\beta}_{LS} - \beta)^T (\hat{\beta}_{LS} - \beta) + \lambda|\beta| \Big) \\ &= \arg\min_{\beta_1, \ldots, \beta_p} \sum_{i=1}^{p} \Big((\beta_j - \hat{\beta}_{(LS)j})^2 + \lambda|\beta_j| \Big) \end{aligned}$$

Therefore, to solve the one-dim lasso above, define

$$f(x) = (x - a)^2 + \lambda|x|,\ a \in \mathbb{R},\ \lambda > 0$$

The value x^* that minimizes $f(x)$ must satisfy:

$$\frac{\partial}{\partial x}(x^* - a)^2 + \lambda \frac{\partial}{\partial x}|x^*| = 0$$
$$2(x^* - a) + \lambda z^* = 0$$

where z^* is the *sub-gradient* of the absolute value function evaluated at x^*, which equals to $sign(x^*)$ if $x^* \neq 0$ and any number in [-1,1], if $x^* = 0$.

The unique minimizer is the **soft-thresholding operator**

$$x^* = S_{\lambda/2}(a) = \text{sign}(a)(|a| - \lambda/2)_+ = \begin{cases} a - \lambda/2 & \text{if } a > \lambda/2, \\ 0 & \text{if } |a| \leq \lambda/2, \\ a + \lambda/2 & \text{if } a < -\lambda/2. \end{cases}$$

Consequently the *orthonormal lasso solution* is simply

$$\hat{\beta}_j^{\text{lasso}} = S_{\lambda/2}\Big(\hat{\beta}_j^{\text{LS}}\Big).$$

A sufficiently large λ forces some coefficients to zero, so the lasso simultaneously selects variables and shrinks the retained coefficients.

Remarks

- Lasso is especially attractive when the underlying signal is *sparse*, since lasso will "make" some of the β coefficients zero keeping the coefficients that will have an effect on $\mathbf{y}$. This is the reason why this method also works when the number of predictors is larger than the sample size ($p > n$). These are scenarios often encountered in genomic or proteomic data where the design matrices tend to have a lot of zeros and too many predictors.
- In lasso as in ridge regression, we can select λ using Cross-Validation (CV). When λ increases, the number of predictors decreases.

Comparing Ridge Regression and Lasso

Ridge regression performs proportional shrinkage and tends to work well when many predictors have moderate effects of similar size. Lasso performs both shrinkage and selection and is preferable when only a few predictors have large effects and the rest are essentially zero. Because the true sparsity pattern is rarely known in advance, cross-validation is the practical tool for deciding which method is better for a given data set.

Geometrically, the residual-sum-of-squares contours are ellipses centered at the ordinary-least-squares estimate. Ridge regression finds the first point where these ellipses touch the disk $\beta^\top \beta \leq t$; lasso finds the first point where they

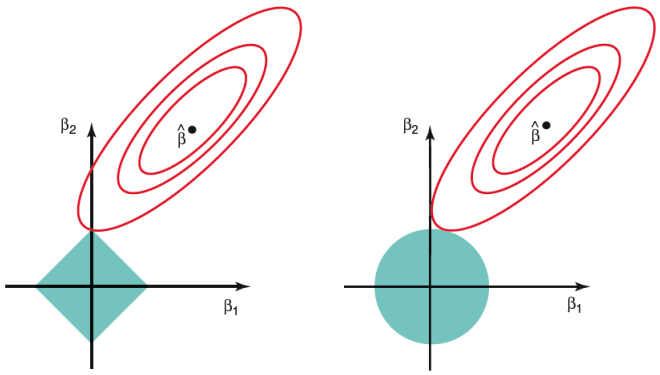


Figure 3.2: Contour of the optimization for Lasso (left) and Ridge (right).

touch the diamond $\|\beta\|_1 \leq t$. The corners of the diamond make exact zeros possible.

In the picture below, we can see the difference in the contours of optimization for lasso (left) and Ridge (right) when there are only two parameters. The residual sum of squares has elliptical contours, centered at the full least squares estimate, $\hat{\beta}_{LS}$. The constraint region for ridge regression is the disk $\beta_1^2 + \beta_2^2 \leq t$, while that for lasso is the diamond $|\beta_1| + |\beta_2| \leq t$. Both methods find the first point where the elliptical contours hit the constraint region. Unlike the disk, the diamond has corners which means that if the solution occurs at a corner, then it has one parameter β_j equal to zero. When $p > 2$, the diamond becomes a rhomboid, and has many corners, flat edges and faces; there are many more opportunities for the estimated parameters to be zero.

Lasso with $p > n$

- When $\mathbf{X}$ has full column rank the lasso criterion is strictly convex, so the minimizer is unique.
- When $\mathbf{X}$ is rank-deficient (in particular when $p > n$) the criterion remains convex but is no longer strictly convex; consequently the solution need not be unique. In the $p > n$ regime the lasso selects at most n variables before saturating—an intrinsic limitation of the convex optimization problem.

3.4 The `Student Performance` Example

Up to now, we studied two complementary approaches to controlling model complexity in linear regression. Classical subset-selection methods (best-subset,

forward, backward, and stepwise) explicitly search over candidate models and penalize complexity via information criteria such as AIC, BIC, adjusted R^2, or Mallows' C_p.

When the number of predictors is large, continuous shrinkage methods become preferable. Ridge regression (ℓ_2 penalty) produces a closed-form estimator that shrinks all coefficients toward zero, with stronger shrinkage applied to low-variance principal-component directions; it is especially useful under multicollinearity but does not perform variable selection. Lasso regression (ℓ_1 penalty) simultaneously shrinks coefficients and sets some of them exactly to zero, thereby combining estimation and variable selection; it is particularly effective when the underlying signal is sparse or when $p > n$. Both ridge and lasso require the predictors to be centered and scaled, and the tuning parameter λ is typically chosen by cross-validation.

The choice between these methods ultimately depends on the *bias–variance trade-off* and on whether sparsity is expected in the true model.

In this final section we illustrate the practical implementation of the variable-selection and shrinkage methods developed earlier, using a real data example.

The analysis is based on the `student-mat.csv` data set, which contains information on student achievement in secondary education at two Portuguese schools. The original study is described in:

> "Using data mining to predict secondary school student performance" By P. Cortez, A. M. G. Silva. 2008. Published in Proceedings of 5th Annual Future Business Technology Conference.

The data are publicly available from the UCI Machine Learning Repository: here

The analysis of this data set is available in R and Python.

Chapter 4

NonLinear Regression

As we discussed in the previous sections, multiple linear regression relies on a set of assumptions about the error terms (normality, homoscedasticity, and independence) together with the fundamental assumption that the mean response is a linear function of the predictors. When one or more of these assumptions are violated, the applicability of the linear model is limited. In this section we focus on **relaxing the linearity assumption** and introduce three of the most widely used approaches to nonlinear regression:

(i) Polynomial regression
(ii) Regression splines
(iii) Smoothing splines

A Note on Nonlinearity

Assume that the true underlying model takes the form

$$Y = f(\mathbf{X}) + \varepsilon$$

where ε satisfies the usual assumptions. So far we have **approximated** the *unknown* function f with a multiple linear regression model:

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + ... + \beta_p X_{ip} + \varepsilon_i$$

This model is linear both in the predictors X_{ij} and in the coefficients β_j.

Now consider the following two models:

$$Y_i = \beta_1 X_{i1} + \beta_2 X_{i1}^2 + \beta_3 X_{i2} + \beta_4 X_{i2}^2 + \beta_5 X_{i1} X_{i2} + \varepsilon_i$$

or

$$\log_{10} Y_i = \beta_1 X_{i1} + \beta_2 \sqrt{X_{i1}} + \beta_3 e^{X_{i3}} + \varepsilon_i$$

Both are nonlinear in the predictors but remain linear in the coefficients β_j. In contrast, the model

$$Y_i = \frac{\gamma_0}{1 + \gamma_1 e^{\gamma_2 X_i}} + \varepsilon_i$$

is nonlinear in both the parameters *and* the predictors.

In this chapter we approximate the general nonlinear relationship f using models that are nonlinear in the predictors (so that we can capture nonlinear associations between the response and the features) yet linear in the coefficients. Equivalently, these models can be viewed as ordinary linear regressions in which the original predictors have been replaced by nonlinear transformations of those predictors.

The main advantage of this approach is that we can often describe nonlinear relationships effectively while still estimating the parameters easily. Because the models remain linear in the parameters, the coefficients continue to be obtained by ordinary least squares, which again reduces to solving a system of linear equations.

4.1 Polynomial Regression

The simplest form of nonlinear regression is polynomial regression, which extends the linear model by including higher-order powers of the predictor(s). To study this approach we first introduce polynomial basis functions.

4.1.1 Polynomial Basis Functions

If $b_j(x)$ denotes the jth basis function, then any function f that lies in the span of these basis functions can be written

$$f(x) = \sum_{j=0}^{d} b_j(x)\beta_j$$

for suitable coefficients β_j. Consequently the nonlinear model $y_i = f(x_i) + \varepsilon_i$ becomes a linear model in the coefficients:

$$y_i = \beta_0 + \sum_{j=1}^{d} b_j(x_i)\beta_j + \varepsilon_i$$

Suppose we believe that f is a polynomial of degree at most 4. A natural basis for the space of all such polynomials is

$$
\begin{aligned}
b_0(x) &= 1 \\
b_1(x) &= x \\
b_2(x) &= x^2 \\
b_3(x) &= x^3 \\
b_4(x) &= x^4
\end{aligned}
$$

The corresponding model is therefore

$$
y_i = \underbrace{\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \beta_4 x_i^4}_{=\beta_0 + \sum_{j=1}^{d} b_j(x_i)\beta_j} + \varepsilon_i
$$

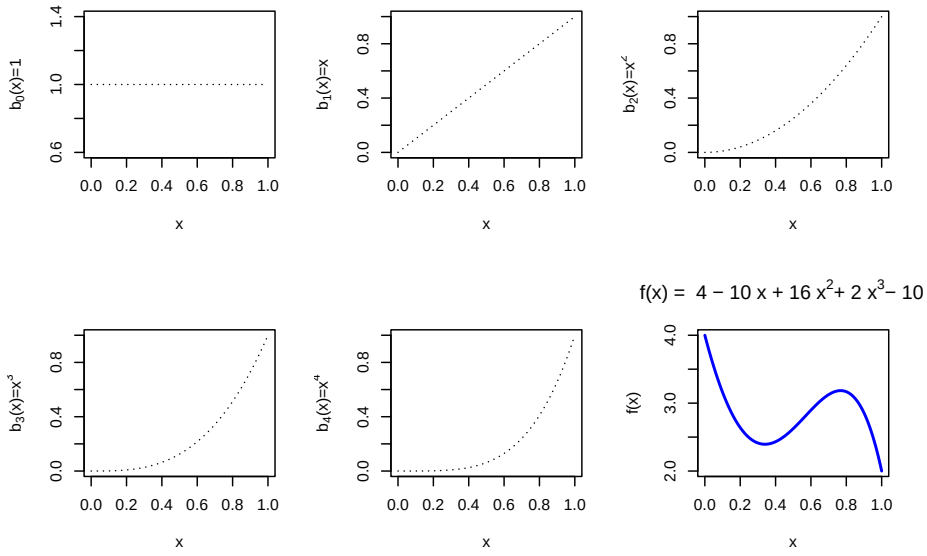
Illustration of the Polynomial Basis Functions

The following code plots the individual polynomial basis functions of degree 0 through 4 and a linear combination of them.

```
x = seq(0, 1, by = 0.001)
b0 = rep(1, length(x))
b1 = x
b2 = x^2
b3 = x^3
b4 = x^4

fun1 = 4 * b0 - 10 * b1 + 16 * b2 + 2 * b3 - 10 * b4

par(mfrow = c(2, 3))
plot(x, b0, type = "l", lty = 3, ylab = expression("b"[0] * "(x)=1"))
plot(x, b1, type = "l", lty = 3, ylab = expression("b"[1] * "(x)=x"))
plot(x, b2, type = "l", lty = 3, ylab = expression("b"[2] * "(x)=x"^2))
plot(x, b3, type = "l", lty = 3, ylab = expression("b"[3] * "(x)=x"^3))
plot(x, b4, type = "l", lty = 3, ylab = expression("b"[4] * "(x)=x"^4))
plot(x, fun1, type = "l", ylab = "f(x)", main = expression("f(x) =  4 - 10 x + 16 x"^2 *
    "+ 2 x"^3 * "- 10 x"^4), col = "blue", lwd = 2)
```



4.1.2 Polynomial Regression

From now on we assume that the predictor x is one-dimensional; the multivariate case will be discussed later. For $x_i \in \mathbb{R}$ the polynomial regression model of degree d is

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \varepsilon_i$$

We simply create the new variables $X_2 = X^2, \ldots, X_d = X^d$ and treat the problem as ordinary multiple linear regression:

$$\underset{n\times 1}{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}} = \underset{n\times(d+1)}{\begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix}} \underset{(d+1)\times 1}{\begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_d \end{pmatrix}} + \varepsilon$$

Polynomial Regression Model

A *non-linear* mean function can be represented with a polynomial basis as

$$y_i = f(x_i) + \varepsilon_i \quad \longrightarrow \quad y_i = \beta_0 + \sum_{j=1}^{d} b_j(x_i)\beta_j + \varepsilon_i$$

where d is the degree of the polynomial.

How do we choose d?

1. *Forward Approach*: Begin with a linear model and successively add higher-order terms until the most recently added term is no longer statistically significant.

2. *Backward Approach*: Start with a high-degree polynomial and successively remove the highest-order term that is not statistically significant.

Once a degree d has been selected, we ordinarily do not test the significance of the lower-order terms. In other words, a polynomial of degree d is understood to contain all powers up to d.

Reasoning

In regression we prefer results that are invariant to simple location or scale changes of the predictor. Consider the following illustration. Suppose the data $\{(y_i, x_i)\}_{i=1}^n$ are generated by

$$y_i = x_i^2 + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

If the predictor is recorded instead as $z_i = x_i + 2$, the same relationship becomes

$$y_i = (z_i - 2)^2 + \varepsilon_i = 4 - 4z_i + z_i^2 + \varepsilon_i.$$

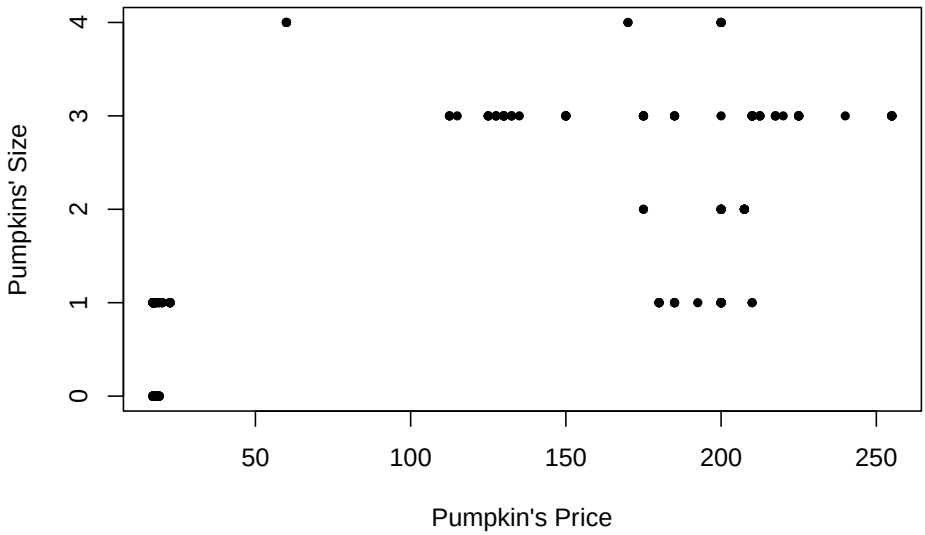
A linear term that was absent in the original scale now appears. Consequently, once we decide that a degree-d polynomial is appropriate we retain every lower-order term.

Exception: If scientific knowledge dictates a specific functional form (for example, a pure quadratic relationship $Y \approx X^2 + C$ arising from a physical law), then it is legitimate to test whether the lower-order terms may be omitted.

The Chicago Pumpkins Example

The file `pumpkins.csv` contains information regarding the `size` and `price` of pumpkins sold in the Chicago area (data can be found here.). Our goal is to *predict* the pumpkin size from price.

```
pumpkins = read.csv("data/week4/chicagopumpkins.csv", header = TRUE)
plot(pumpkins$price, pumpkins$size, pch = 20, xlab = "Pumpkin's Price",
    ylab = "Pumpkins' Size")
```



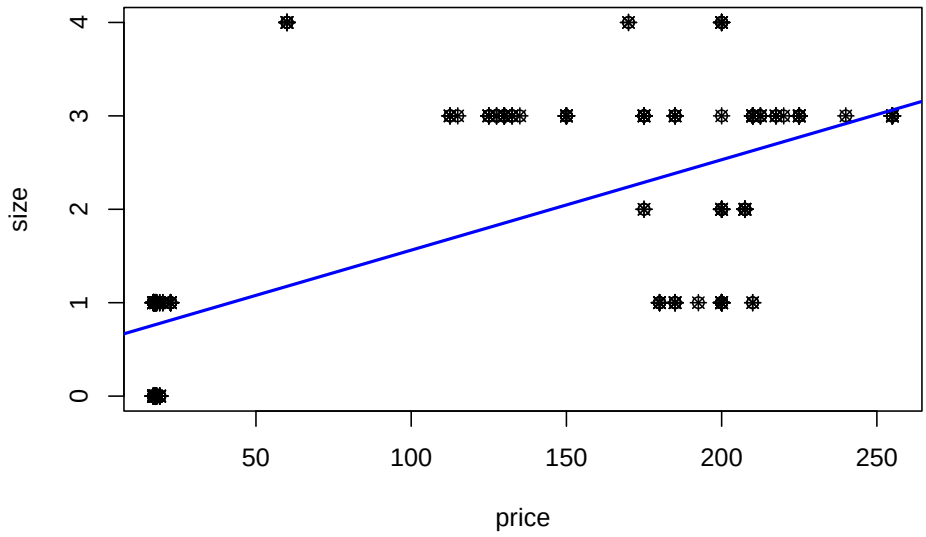
A linear fit is visibly inadequate:

```
lm.pumpkins = lm(size ~ price, data = pumpkins)
summary(lm.pumpkins)
```

```
##
## Call:
## lm(formula = size ~ price, data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.6272 -0.7594  0.2244  0.3728  2.8245
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 0.5948315  0.0988339   6.018 6.35e-09 ***
## price       0.0096781  0.0006856  14.117  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9477 on 246 degrees of freedom
## Multiple R-squared:  0.4475, Adjusted R-squared:  0.4453
## F-statistic: 199.3 on 1 and 246 DF,  p-value: < 2.2e-16
```

Although price is statistically significant, R^2 is modest and the scatter plot does not support a straight-line relationship.

```
plot(size ~ price, data = pumpkins)
points(size ~ price, data = pumpkins, pch = 8)
abline(lm.pumpkins, col = "blue", lwd = 2)
```



We therefore consider higher-order polynomials, first by forward selection and then by backward elimination.

(1) Forward selection. We start with a linear model and add successive powers until the newest term becomes insignificant.

```
# Forward Selection
lm.pumpkins = lm(size ~ price, data = pumpkins)
summary(lm.pumpkins)
```

```
##
## Call:
## lm(formula = size ~ price, data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.6272 -0.7594  0.2244  0.3728  2.8245
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 0.5948315  0.0988339   6.018 6.35e-09 ***
## price       0.0096781  0.0006856  14.117  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9477 on 246 degrees of freedom
## Multiple R-squared:  0.4475, Adjusted R-squared:  0.4453
## F-statistic: 199.3 on 1 and 246 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins2 = lm(size ~ price + I(price^2), data = pumpkins)
summary(lm.pumpkins2)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2), data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.6438 -0.6029  0.3695  0.4895  2.3941
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.093e-01  1.174e-01   0.932    0.353
## price        3.037e-02  3.208e-03   9.469  < 2e-16 ***
## I(price^2)  -9.051e-05  1.375e-05  -6.582  2.8e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8754 on 245 degrees of freedom
## Multiple R-squared:  0.5305, Adjusted R-squared:  0.5267
## F-statistic: 138.4 on 2 and 245 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins3 = lm(size ~ price + I(price^2) + I(price^3), data = pumpkins)
summary(lm.pumpkins3)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3), data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.2786 -0.4937 -0.1368  0.5531  1.8633
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.414e+00  1.691e-01   -8.36 4.83e-15 ***
## price        1.256e-01  9.079e-03   13.83  < 2e-16 ***
## I(price^2)  -9.845e-04  8.239e-05  -11.95  < 2e-16 ***
## I(price^3)   2.228e-06  2.034e-07   10.95  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7182 on 244 degrees of freedom
## Multiple R-squared:  0.6853, Adjusted R-squared:  0.6814
## F-statistic: 177.1 on 3 and 244 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins4 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4),
    data = pumpkins)
summary(lm.pumpkins4)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4),
##     data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.3200 -0.4497 -0.1241  0.5539  1.7925
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.871e+00  3.782e-01  -7.590 6.83e-13 ***
## price        2.314e-01  2.630e-02   8.800 2.60e-16 ***
## I(price^2)  -2.565e-03  3.785e-04  -6.776 9.25e-11 ***
## I(price^3)   1.061e-05  1.973e-06   5.378 1.76e-07 ***
## I(price^4)  -1.470e-08  3.443e-09  -4.271 2.80e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6941 on 243 degrees of freedom
## Multiple R-squared:  0.7073, Adjusted R-squared:  0.7025
## F-statistic: 146.8 on 4 and 243 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins5 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4) +
    I(price^5), data = pumpkins)
summary(lm.pumpkins5)
```

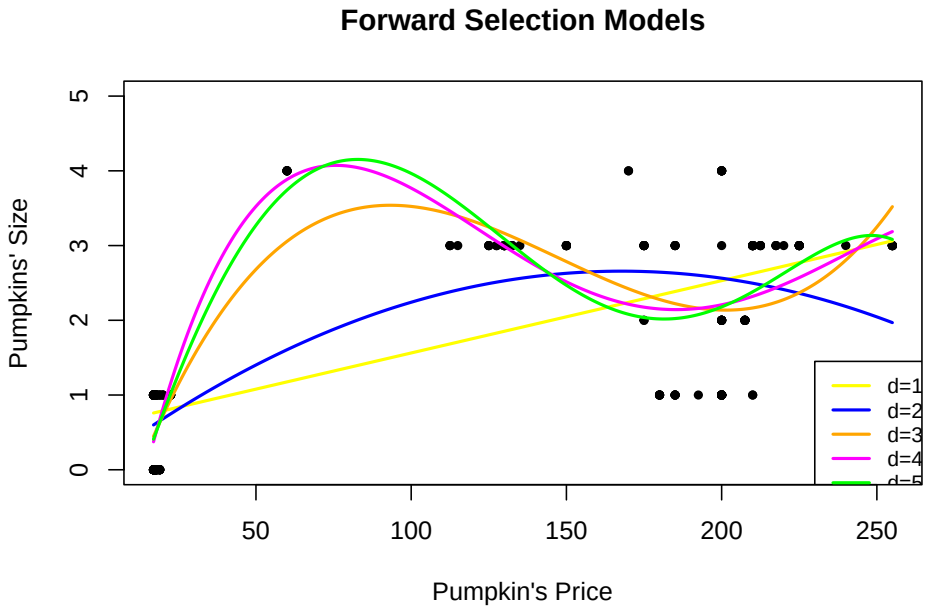
```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4) +
##     I(price^5), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.38082 -0.46537 -0.08211  0.53463  1.91954
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.691e+00  7.112e-01  -2.378   0.0182 *
## price        1.305e-01  5.791e-02   2.253   0.0251 *
## I(price^2)  -2.479e-04  1.244e-03  -0.199   0.8422
## I(price^3)  -1.078e-05  1.113e-05  -0.969   0.3333
```

```
## I(price^4)   7.118e-08  4.409e-08   1.615   0.1077
## I(price^5)  -1.248e-10  6.386e-11  -1.954   0.0519 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6901 on 242 degrees of freedom
## Multiple R-squared:  0.7118, Adjusted R-squared:  0.7059
## F-statistic: 119.6 on 5 and 242 DF,  p-value: < 2.2e-16
```

The fifth-order term has a p-value of 0.0519, which exceeds the conventional 5% threshold. We therefore stop at degree 4. The fitted model is

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x + \hat{\beta}_2 x^2 + \hat{\beta}_3 x^3 + \hat{\beta}_4 x^4$$

```
newprice = data.frame(price = seq(17, 255, 1))
plot(pumpkins$price, pumpkins$size, pch = 20, ylim = c(0, 5),
    xlab = "Pumpkin's Price", ylab = "Pumpkins' Size", main = "Forward Selection Models
lines(newprice$price, predict(lm.pumpkins, newprice), col = "yellow",
    lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins2, newprice), col = "blue",
    lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins3, newprice), col = "orange",
    lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins4, newprice), col = "magenta",
    lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins5, newprice), col = "green",
    lty = 1, lwd = 2)
legend(230, 1.45, legend = c("d=1", "d=2", "d=3", "d=4", "d=5"),
    col = c("yellow", "blue", "orange", "magenta", "green"),
    lty = c(1, 1, 1, 1, 1), cex = 0.8, lwd = c(2, 2, 2, 2, 2))
```



The magenta curve corresponds to the selected degree-4 model.

(2) Backward elimination. We begin with a high-degree polynomial and remove terms until the highest-order coefficient is significant.

```
lm.pumpkins10 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4) +
    I(price^5) + I(price^6) + I(price^7) + I(price^8) + I(price^9) +
    I(price^10), data = pumpkins)
summary(lm.pumpkins10)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4) +
##     I(price^5) + I(price^6) + I(price^7) + I(price^8) + I(price^9) +
##     I(price^10), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.44082 -0.45530 -0.01853  0.53535  2.12546
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.331e+01  2.671e+01   0.498    0.619
## price       -2.191e+00  4.274e+00  -0.513    0.609
## I(price^2)   1.396e-01  2.625e-01   0.532    0.595
## I(price^3)  -4.389e-03  8.140e-03  -0.539    0.590
```

```
## I(price^4)   8.198e-05  1.458e-04   0.562    0.575
## I(price^5)  -9.818e-07  1.624e-06  -0.605    0.546
## I(price^6)   7.731e-09  1.163e-08   0.664    0.507
## I(price^7)  -3.970e-11  5.374e-11  -0.739    0.461
## I(price^8)   1.276e-13  1.549e-13   0.824    0.411
## I(price^9)  -2.321e-16  2.535e-16  -0.916    0.361
## I(price^10)  1.821e-19  1.800e-19   1.012    0.313
##
## Residual standard error: 0.6483 on 237 degrees of freedom
## Multiple R-squared:  0.7509, Adjusted R-squared:  0.7404
## F-statistic: 71.45 on 10 and 237 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins9 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4) +
    I(price^5) + I(price^6) + I(price^7) + I(price^8) + I(price^9),
    data = pumpkins)
summary(lm.pumpkins9)
```

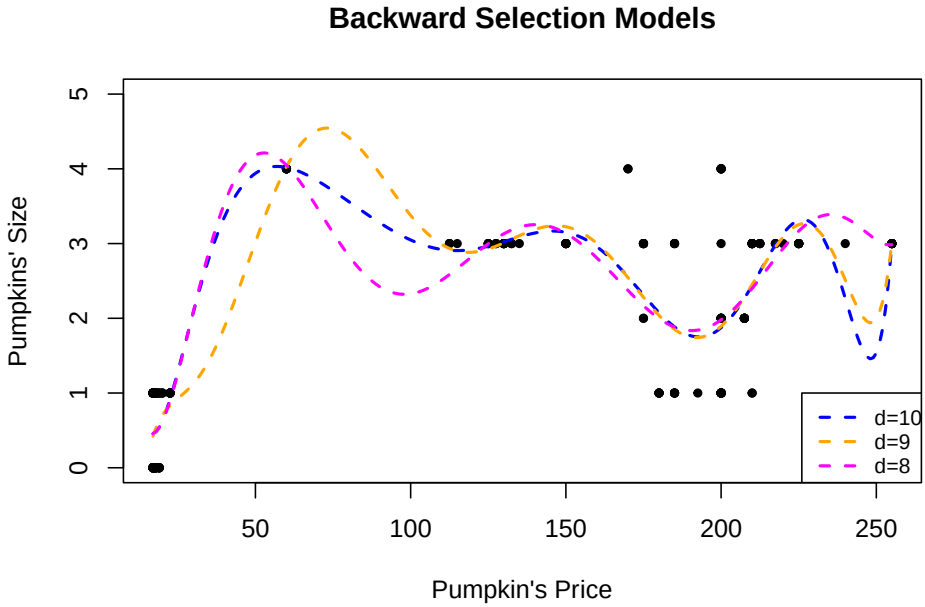
```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4) +
##     I(price^5) + I(price^6) + I(price^7) + I(price^8) + I(price^9),
##     data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.4514 -0.4230 -0.0091  0.5177  2.1072
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.191e+01  9.589e+00  -1.242   0.2154
## price        1.877e+00  1.450e+00   1.295   0.1967
## I(price^2)  -1.125e-01  8.271e-02  -1.360   0.1751
## I(price^3)   3.506e-03  2.316e-03   1.514   0.1314
## I(price^4)  -6.094e-05  3.623e-05  -1.682   0.0939 .
## I(price^5)   6.252e-07  3.391e-07   1.844   0.0664 .
## I(price^6)  -3.875e-09  1.945e-09  -1.993   0.0475 *
## I(price^7)   1.425e-11  6.704e-12   2.125   0.0346 *
## I(price^8)  -2.859e-14  1.276e-14  -2.241   0.0259 *
## I(price^9)   2.411e-17  1.030e-17   2.340   0.0201 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6483 on 238 degrees of freedom
## Multiple R-squared:  0.7499, Adjusted R-squared:  0.7404
## F-statistic: 79.27 on 9 and 238 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins8 = lm(size ~ price + I(price^2) + I(price^3) + I(price^4) +
    I(price^5) + I(price^6) + I(price^7) + I(price^8), data = pumpkins)
summary(lm.pumpkins8)
```

```
##
## Call:
## lm(formula = size ~ price + I(price^2) + I(price^3) + I(price^4) +
##     I(price^5) + I(price^6) + I(price^7) + I(price^8), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.39988 -0.45678 -0.05827  0.53309  2.02119
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  9.140e+00  3.347e+00   2.731 0.006791 **
## price       -1.368e+00  4.257e-01  -3.215 0.001486 **
## I(price^2)   7.522e-02  2.032e-02   3.702 0.000266 ***
## I(price^3)  -1.798e-03  4.783e-04  -3.759 0.000214 ***
## I(price^4)   2.262e-05  6.154e-06   3.675 0.000293 ***
## I(price^5)  -1.611e-07  4.541e-08  -3.549 0.000466 ***
## I(price^6)   6.535e-10  1.918e-10   3.407 0.000771 ***
## I(price^7)  -1.406e-12  4.316e-13  -3.259 0.001282 **
## I(price^8)   1.246e-15  4.008e-16   3.108 0.002112 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6544 on 239 degrees of freedom
## Multiple R-squared:  0.7441, Adjusted R-squared:  0.7355
## F-statistic: 86.87 on 8 and 239 DF,  p-value: < 2.2e-16
```

Starting from degree 10 we arrive at a degree-9 model. The corresponding fits are shown below.

```
newprice = data.frame(price = seq(17, 255, 1))
plot(pumpkins$price, pumpkins$size, ylim = c(0, 5), pch = 20,
    xlab = "Pumpkin's Price", ylab = "Pumpkins' Size", main = "Backward Selection Models")
lines(newprice$price, predict(lm.pumpkins10, newprice), col = "blue",
    lty = 2, lwd = 2)
lines(newprice$price, predict(lm.pumpkins9, newprice), col = "orange",
    lty = 2, lwd = 2)
lines(newprice$price, predict(lm.pumpkins8, newprice), col = "magenta",
    lty = 2, lwd = 2)
legend(226, 1, legend = c("d=10", "d=9", "d=8"), col = c("blue",
    "orange", "magenta"), lty = c(2, 2, 2), cex = 0.8, lwd = 2)
```



The magenta curve is the degree-9 fit selected by backward elimination.

As always, the final model should be subjected to the usual residual diagnostics.

4.1.3 Orthogonal Polynomials

High-degree polynomials are generally not recommended: they tend to be numerically unstable, hard to interpret, and the successive powers $x, x^2, \ldots, x^d$ are highly correlated, producing severe multicollinearity. One remedy is to work with orthogonal polynomials

$$y_i = \beta_0 + \beta_1 z_1 + \ldots + \beta_d z_d + \varepsilon_i$$

where each $ z_j $ is a polynomial of exact degree j constructed so that

$$z_j^\top z_k = 0 \quad \text{for all } j \neq k.$$

The Chicago Pumpkins Example In R the function poly generates orthogonal polynomial bases. The forward- and backward-selection procedures can be repeated with orthogonal polynomials as follows.

```
# Forward Selection
lm.pumpkins02 = lm(size ~ poly(price, 2), data = pumpkins)
summary(lm.pumpkins02)
```

```
##
## Call:
## lm(formula = size ~ poly(price, 2), data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.6438 -0.6029  0.3695  0.4895  2.3941
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1.70161    0.05559  30.612  < 2e-16 ***
## poly(price, 2)1 13.37846    0.87538  15.283  < 2e-16 ***
## poly(price, 2)2 -5.76139    0.87538  -6.582  2.8e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8754 on 245 degrees of freedom
## Multiple R-squared:  0.5305, Adjusted R-squared:  0.5267
## F-statistic: 138.4 on 2 and 245 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins03 = lm(size ~ poly(price, 3), data = pumpkins)
summary(lm.pumpkins03)
```

```
##
## Call:
## lm(formula = size ~ poly(price, 3), data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.2786 -0.4937 -0.1368  0.5531  1.8633
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)       1.7016     0.0456  37.313  < 2e-16 ***
## poly(price, 3)1  13.3785     0.7182  18.628  < 2e-16 ***
## poly(price, 3)2  -5.7614     0.7182  -8.022 4.35e-14 ***
## poly(price, 3)3   7.8672     0.7182  10.954  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7182 on 244 degrees of freedom
## Multiple R-squared:  0.6853, Adjusted R-squared:  0.6814
## F-statistic: 177.1 on 3 and 244 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins04 = lm(size ~ poly(price, 4), data = pumpkins)
summary(lm.pumpkins04)
```

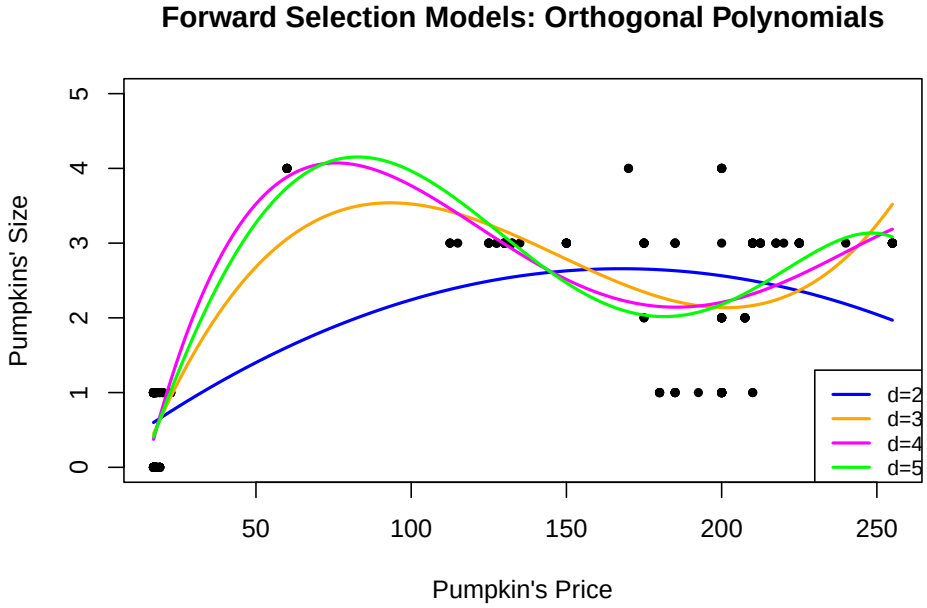
```
##
## Call:
## lm(formula = size ~ poly(price, 4), data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.3200 -0.4497 -0.1241  0.5539  1.7925
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1.70161    0.04407  38.608  < 2e-16 ***
## poly(price, 4)1 13.37846    0.69408  19.275  < 2e-16 ***
## poly(price, 4)2 -5.76139    0.69408  -8.301 7.22e-15 ***
## poly(price, 4)3  7.86718    0.69408  11.335  < 2e-16 ***
## poly(price, 4)4 -2.96423    0.69408  -4.271 2.80e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6941 on 243 degrees of freedom
## Multiple R-squared:  0.7073, Adjusted R-squared:  0.7025
## F-statistic: 146.8 on 4 and 243 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins05 = lm(size ~ poly(price, 5), data = pumpkins)
summary(lm.pumpkins05)
```

```
##
## Call:
## lm(formula = size ~ poly(price, 5), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.38082 -0.46537 -0.08211  0.53463  1.91954
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1.70161    0.04382  38.831  < 2e-16 ***
## poly(price, 5)1 13.37846    0.69009  19.387  < 2e-16 ***
## poly(price, 5)2 -5.76139    0.69009  -8.349 5.35e-15 ***
## poly(price, 5)3  7.86718    0.69009  11.400  < 2e-16 ***
## poly(price, 5)4 -2.96423    0.69009  -4.295 2.53e-05 ***
## poly(price, 5)5 -1.34841    0.69009  -1.954   0.0519 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6901 on 242 degrees of freedom
## Multiple R-squared:  0.7118, Adjusted R-squared:  0.7059
```

```
## F-statistic: 119.6 on 5 and 242 DF,  p-value: < 2.2e-16
```

```
newprice = data.frame(price = seq(17, 255, 1))
plot(pumpkins$price, pumpkins$size, pch = 20, ylim = c(0, 5),
    xlab = "Pumpkin's Price", ylab = "Pumpkins' Size", main = "Forward Selection Models: Orthogon
lines(newprice$price, predict(lm.pumpkins02, newprice), col = "blue",
    lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins03, newprice), col = "orange",
    lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins04, newprice), col = "magenta",
    lty = 1, lwd = 2)
lines(newprice$price, predict(lm.pumpkins05, newprice), col = "green",
    lty = 1, lwd = 2)
legend(230, 1.3, legend = c("d=2", "d=3", "d=4", "d=5"), col = c("blue",
    "orange", "magenta", "green"), lty = c(1, 1, 1, 1), cex = 0.8,
    lwd = c(2, 2, 2, 2))
```




```
# Backward Selection
lm.pumpkins010 = lm(size ~ poly(price, 10), data = pumpkins)
summary(lm.pumpkins010)
```

```
##
## Call:
## lm(formula = size ~ poly(price, 10), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```

```
## -1.44082 -0.45530 -0.01853  0.53535  2.12546
##
## Coefficients:
##                    Estimate Std. Error t value Pr(>|t|)
## (Intercept)         1.70161    0.04117  41.335  < 2e-16 ***
## poly(price, 10)1   13.37846    0.64829  20.636  < 2e-16 ***
## poly(price, 10)2   -5.76139    0.64829  -8.887  < 2e-16 ***
## poly(price, 10)3    7.86718    0.64829  12.135  < 2e-16 ***
## poly(price, 10)4   -2.96423    0.64829  -4.572 7.77e-06 ***
## poly(price, 10)5   -1.34841    0.64829  -2.080  0.03861 *
## poly(price, 10)6   -1.45456    0.64829  -2.244  0.02578 *
## poly(price, 10)7   -2.57955    0.64829  -3.979 9.20e-05 ***
## poly(price, 10)8    2.03378    0.64829   3.137  0.00192 **
## poly(price, 10)9    1.51698    0.64829   2.340  0.02012 *
## poly(price, 10)10   0.65591    0.64829   1.012  0.31269
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6483 on 237 degrees of freedom
## Multiple R-squared:  0.7509, Adjusted R-squared:  0.7404
## F-statistic: 71.45 on 10 and 237 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins09 = lm(size ~ poly(price, 9), data = pumpkins)
summary(lm.pumpkins09)
```

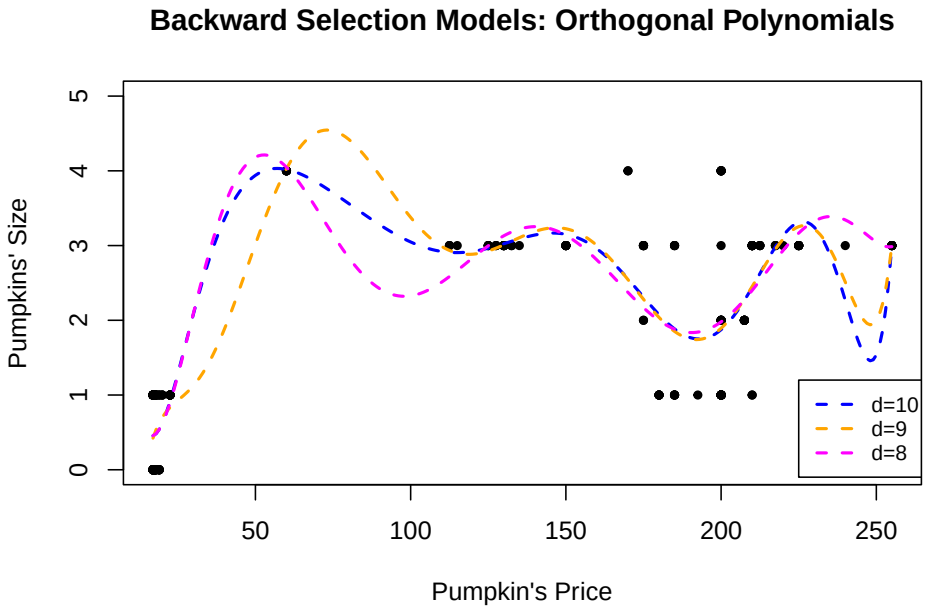
```
##
## Call:
## lm(formula = size ~ poly(price, 9), data = pumpkins)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.4514 -0.4230 -0.0091  0.5177  2.1072
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)        1.70161    0.04117  41.333  < 2e-16 ***
## poly(price, 9)1   13.37846    0.64833  20.635  < 2e-16 ***
## poly(price, 9)2   -5.76139    0.64833  -8.887  < 2e-16 ***
## poly(price, 9)3    7.86718    0.64833  12.135  < 2e-16 ***
## poly(price, 9)4   -2.96423    0.64833  -4.572 7.76e-06 ***
## poly(price, 9)5   -1.34841    0.64833  -2.080  0.03861 *
## poly(price, 9)6   -1.45456    0.64833  -2.244  0.02578 *
## poly(price, 9)7   -2.57955    0.64833  -3.979 9.20e-05 ***
## poly(price, 9)8    2.03378    0.64833   3.137  0.00192 **
## poly(price, 9)9    1.51698    0.64833   2.340  0.02012 *
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6483 on 238 degrees of freedom
## Multiple R-squared:  0.7499, Adjusted R-squared:  0.7404
## F-statistic: 79.27 on 9 and 238 DF,  p-value: < 2.2e-16
```

```
lm.pumpkins08 = lm(size ~ poly(price, 8), data = pumpkins)
summary(lm.pumpkins08)
```

```
##
## Call:
## lm(formula = size ~ poly(price, 8), data = pumpkins)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.39988 -0.45678 -0.05827  0.53309  2.02119
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1.70161    0.04155  40.951  < 2e-16 ***
## poly(price, 8)1 13.37846    0.65437  20.445  < 2e-16 ***
## poly(price, 8)2 -5.76139    0.65437  -8.805 2.71e-16 ***
## poly(price, 8)3  7.86718    0.65437  12.023  < 2e-16 ***
## poly(price, 8)4 -2.96423    0.65437  -4.530 9.32e-06 ***
## poly(price, 8)5 -1.34841    0.65437  -2.061 0.040421 *
## poly(price, 8)6 -1.45456    0.65437  -2.223 0.027162 *
## poly(price, 8)7 -2.57955    0.65437  -3.942 0.000106 ***
## poly(price, 8)8  2.03378    0.65437   3.108 0.002112 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6544 on 239 degrees of freedom
## Multiple R-squared:  0.7441, Adjusted R-squared:  0.7355
## F-statistic: 86.87 on 8 and 239 DF,  p-value: < 2.2e-16
```

```
plot(pumpkins$price, pumpkins$size, pch = 20, ylim = c(0, 5),
    xlab = "Pumpkin's Price", ylab = "Pumpkins' Size", main = "Backward Selection Models: Orthogo
lines(newprice$price, predict(lm.pumpkins010, newprice), col = "blue",
    lty = 2, lwd = 2)
lines(newprice$price, predict(lm.pumpkins09, newprice), col = "orange",
    lty = 2, lwd = 2)
lines(newprice$price, predict(lm.pumpkins08, newprice), col = "magenta",
    lty = 2, lwd = 2)
legend(225, 1.2, legend = c("d=10", "d=9", "d=8"), col = c("blue",
    "orange", "magenta"), lty = c(2, 2, 2), cex = 0.8, lwd = 2)
```



4.1.4 Piece-wise Polynomials

When the true mean function $\mathbb{E}(Y \mid X = x) = f(x)$ is highly oscillatory, a single high-degree polynomial may be required. Such polynomials are often undesirable. An attractive alternative is to use piecewise polynomials:

1. Partition the range of x into contiguous intervals.
2. Inside each interval approximate f by a low-degree polynomial (typically quadratic or cubic) whose coefficients are allowed to differ across intervals.
3. Impose continuity (and possibly continuity of derivatives) at the knots that separate the intervals.

The resulting fit is sometimes called *broken-stick* regression when the pieces are linear. The principal advantage is that each observation influences the fit only locally; the principal disadvantage is that the overall curve is usually less smooth than a single global polynomial of comparable complexity.

4.2 Splines Regression

Splines are piecewise polynomials thar are constructed so that thay can be both sensitive and smooth, but also capture local features of the data.

A **Cubic Spline** is a curve constructed from sections of cubic polynomials, joined together so that the curve is *continuous up to second derivative*. The points at which the sections join are called the **knots** of the spline. For a conventional spline, the knots occur wherever there is a datum, but for the regression splines the locations of the knots must be chosen. Typically, the knots would either be *evenly spaced* through the range of observed x values, or placed at the *quantiles* of the distribution of unique x values. Each section of cubic has different coefficients, but at the knots *it will match its neighboring sections in value and first two derivatives.*

Cubic Splines (Mathematical) Definition

A function g defined on $[a, b]$ is a **cubic spline** with respect to **knots** $\{\xi_i\}_{i=1}^m$ (specifically $a < \xi_1 < \xi_2 < ... < \xi_m < b$) if:

- g is a cubic polynomial in each of the $m + 1$ intervals, i.e.

$$g(x) = d_i x^3 + c_i x^2 + b_i x + a_i, \quad x \in [\xi_i, \xi_{i+1}]$$

  where $i = 0, ..., m$, $\xi_0 = a$ and $\xi_{m+1} = b$.

- g is continuous up to the *2nd* derivative. Since g is continuous up to the *2nd* derivative *for any point inside an interval*, it suffices to check the following conditions:

$$g^{(0,1,2)}(\xi_i^+) = g^{(0,1,2)}(\xi_i^-), \quad i = 1, ..., m$$

  This expression indicates that the function and the first and second order derivatives are continuous at the knots.

How many free parameters do we need to represent a cubic spline?

(+) 4 parameters (d_i, c_i, b_i, a_i) *for each of the* $(m + 1)$ *intervals.*

(-) 3 constraints *at each of the* m *knots* (continuity constraints).

The total number of free parameters (similar to the number of degrees of freedom) is:

$$4(m + 1) - 3m = m + 4$$

A property of the cubic splines

Suppose the knots $\{\xi_i\}_{i=1}^m$ are given.

- If $g_1(x)$ and $g_2(x)$ are cubic splines, the linear combination

$$a_1 g_1(x) + a_2 g_2(x)$$

  is also a cubic spline, where a_1 and a_2 are known constants.

That is, for a set of given knots, the corresponding *cubic splines form a linear space (of functions) with dim* $(m+4)$.

4.2.1 Examples of Cubic Splines Basis

1. A set of basis functions for cubic splines (w.r.t knots $\{\xi_i\}_{i=1}^m$) is given by:

$$\begin{aligned} h_0(x) &= 1 \\ h_1(x) &= x \\ h_2(x) &= x^2 \\ h_3(x) &= x^3; \\ h_{i+3}(x) &= (x-\xi_i)_+^3, \quad i = 1, 2, \ldots, m \end{aligned}$$

   That is, any cubic spline can be uniquely expressed as:

$$\beta_0 + \sum_{j=1}^{m+3} \beta_j h_j(x)$$

2. Given knot locations, there are many alternative, but equivalent ways of writing down a basis for cubic splines. For example, *another* basis for cubic splines can be written as:

$$\begin{aligned} h_0(x) &= 1 \\ h_1(x) &= x \\ h_{i+1}(x) &= R(x, \xi_i^*), \ i = 1, \ldots, q-1 \end{aligned}$$

   where

$$\begin{aligned} R(x,z) = &\left[(z-1/2)^2 - 1/12\right]\left[(x-1/2)^2 - 1/12\right]/4 \\ &- \left[(|x-z|-1/2)^4 - 1/2(|x-z|-1/2)^2 + 7/240\right]/24 \end{aligned}$$

An Example of a Cubic Splines Basis

We first define the function $R(x,z)$ as above in `R`:

```
R_xz <- function(x, z) {
    ((z - 0.5)^2 - 1/12) * ((x - 0.5)^2 - 1/12)/4 - ((abs(x -
        z) - 0.5)^4 - (abs(x - z) - 0.5)^2/2 + 7/240)/24
}
```

We then need to define the knots

```
new.knots = c(1/6, 3/6, 5/6)
```

In regression examples the x variable will be the predictor (there is no need to generate any x values). Here, we need a "generic" x for illustration purposes.

```
x = seq(0, 1, by = 0.01)
```

Finally, we defined the splines functions, based on the definition given above:

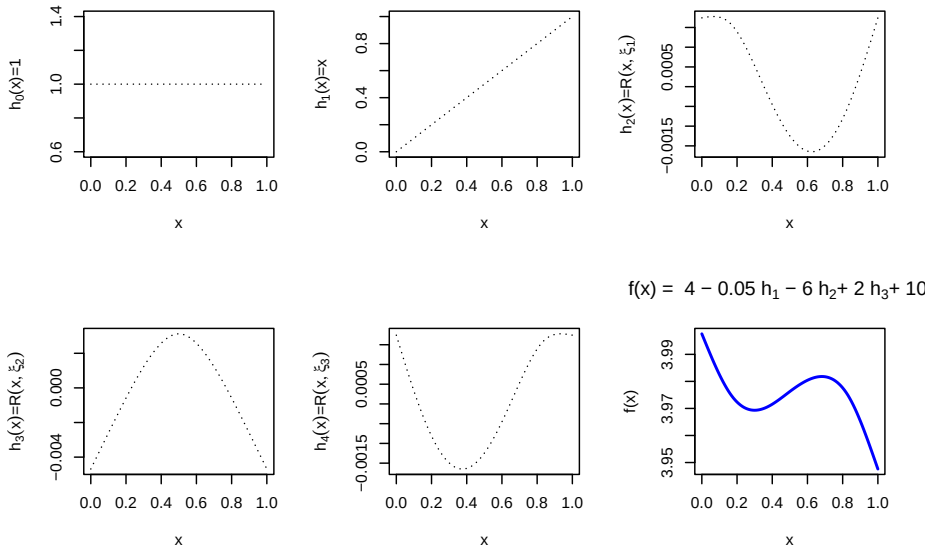
```
spline1 = rep(1, length(x))  # First basis function
spline2 = x  # Second basis function
spline3 = R_xz(x, new.knots[1])  # Third basis function
spline4 = R_xz(x, new.knots[2])  # Fourth basis function
spline5 = R_xz(x, new.knots[3])  # Fifth basis function
```

If we want to represent a function using the basis above, we have:

```
fun1s = 4 * spline1 - 0.05 * spline2 - 6 * spline3 + 2 * spline4 +
    10 * spline5
```

For illustration purposes, we plot the basis functions defined above as well as the function f:

```
par(mfrow = c(2, 3))
plot(x, spline1, type = "l", lty = 3, ylab = expression("h"[0] *
    "(x)=1"))
plot(x, spline2, type = "l", lty = 3, ylab = expression("h"[1] *
    "(x)=x"))
plot(x, spline3, type = "l", lty = 3, ylab = expression(paste(h[2](x)) *
    "=" * paste(R(x, xi[1]))))
plot(x, spline4, type = "l", lty = 3, ylab = expression(paste(h[3](x)) *
    "=" * paste(R(x, xi[2]))))
plot(x, spline5, type = "l", lty = 3, ylab = expression(paste(h[4](x)) *
    "=" * paste(R(x, xi[3]))))
plot(x, fun1s, type = "l", ylab = "f(x)", main = expression("f(x) =  4 - 0.05 h"[1] *
    " - 6 h"[2] * "+ 2 h"[3] * "+ 10 h"[4]), col = "blue", lwd = 2)
```



4.2.2 B-Splines Basis Functions in R

In this course one of the cubic spline bases we will use is the **B-spline** basis.

Cubic B Splines Definition

The cubic B-spline basis on an interval $ [a,b] $ with interior knots $ { *i*}{i=1}^m $ satisfies the following properties:

1. Each basis function is nonzero only on an interval spanned by four successive knots (and is zero elsewhere).
2. On every sub-interval between consecutive knots the basis function is a cubic polynomial.
3. The basis function is continuous and has continuous first and second derivatives at every knot.
4. The basis function integrates to 1 over its support.
5. The two boundary basis functions are modified so that the required continuity conditions hold at the endpoints a and b.

These bases are provided by the `splines` library in `R`; the generating function is `bs()`.

Its **main arguments** are

- `x`: the predictor vector.
- `df`: desired degrees of freedom (number of columns of the returned basis matrix).

- `knots`: vector of interior knot locations.
- `degree`: degree of the piecewise polynomial (default = 3, i.e., cubic).
- `intercept`: if `TRUE` an intercept column is included in the basis; default is `FALSE`. (This is not the intercept of the eventual regression model.)
- `Boundary.knots`: the two boundary knots at which the basis is anchored; by default they are set to the range of the non-missing values of `x`

(The full documentation is available here.)

In practice we supply either the vector knots or the integer df, but not both. All other arguments are left at their defaults.

The value returned by `bs()` is a matrix with `length(x)` rows.

- If df is supplied the matrix has df columns.
- If knots is supplied the number of columns equals

$$\text{length(knots)} + \text{degree} + \text{as.integer(intercept)}.$$

With the default `intercept = FALSE` this reduces to length(knots) + 3.

Specifying knots versus degrees of freedom**

If the knot locations are known, pass them via the `knots` argument:

Defining `knots` location in `bs()`

```
library(splines)
x = seq(0, 1, by = 0.01)  # generic `x` for illustration purposes
new.knots = c(1/6, 3/6, 5/6)  # define three knots at locations 1/6, 3/6, 5/6.
Bsplines.basis1 = bs(x, knots = new.knots)
head(Bsplines.basis1)
```

```
##             1         2         3 4 5 6
## [1,] 0.000000 0.0000000 0.0000000 0 0 0
## [2,] 0.165912 0.0034896 0.0000144 0 0 0
## [3,] 0.304896 0.0135168 0.0001152 0 0 0
## [4,] 0.418824 0.0294192 0.0003888 0 0 0
## [5,] 0.509568 0.0505344 0.0009216 0 0 0
## [6,] 0.579000 0.0762000 0.0018000 0 0 0
```

In this case, `R` sets

$$\text{df} = \text{length(knots)} + \text{degree}$$

(or one more if **if `intercept=TRUE`**).

- If a target degrees of freedom is preferred, pass `df`. Then `bs()` places df - degree interior knots at suitable quantiles of `x` (or `df - degree - 1` knots when `intercept = TRUE`).

Specifying degrees of freedom with `bs()`

```
x = seq(0, 1, by = 0.01)  # generic `x` for illustration purposes
Bsplines.basis2 = bs(x, df = 4)
head(Bsplines.basis2)
```

```
##              1        2        3 4
## [1,] 0.000000 0.000000 0.000000 0
## [2,] 0.058214 0.000592 0.000002 0
## [3,] 0.112912 0.002336 0.000016 0
## [4,] 0.164178 0.005184 0.000054 0
## [5,] 0.212096 0.009088 0.000128 0
## [6,] 0.256750 0.014000 0.000250 0
```

The following code illustrates the first six B-spline basis functions obtained from the three knots defined above.

Illustration of B Splines Basis Function

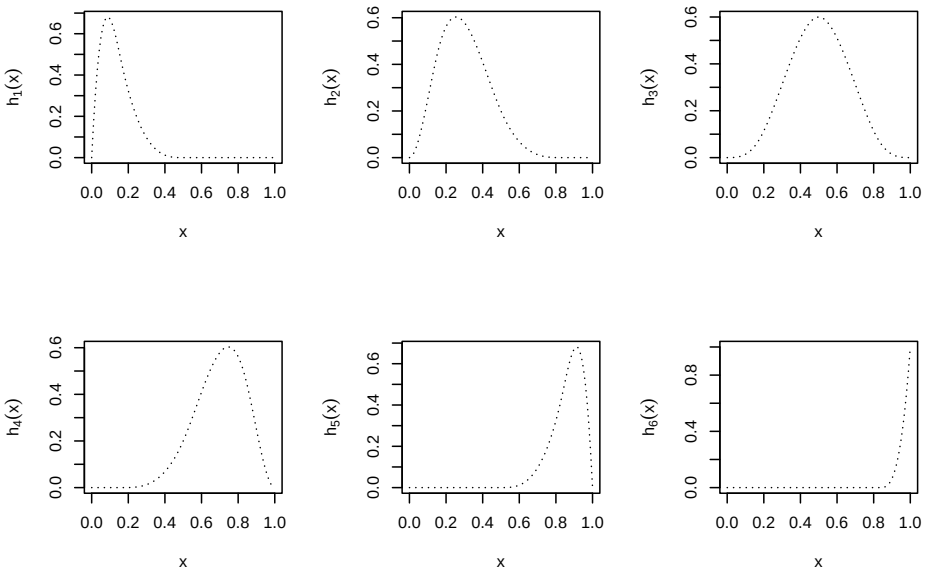
```
Bsplines.basis = bs(x, knots = new.knots)
dim(Bsplines.basis)
```

```
## [1] 101   6
```

```
head(Bsplines.basis)
```

```
##              1         2         3 4 5 6
## [1,] 0.000000 0.0000000 0.0000000 0 0 0
## [2,] 0.165912 0.0034896 0.0000144 0 0 0
## [3,] 0.304896 0.0135168 0.0001152 0 0 0
## [4,] 0.418824 0.0294192 0.0003888 0 0 0
## [5,] 0.509568 0.0505344 0.0009216 0 0 0
## [6,] 0.579000 0.0762000 0.0018000 0 0 0
```

```
par(mfrow = c(2, 3))
plot(x, Bsplines.basis[, 1], type = "l", lty = 3, ylab = expression(paste(h[1](x))))
plot(x, Bsplines.basis[, 2], type = "l", lty = 3, ylab = expression(paste(h[2](x))))
plot(x, Bsplines.basis[, 3], type = "l", lty = 3, ylab = expression(paste(h[3](x))))
plot(x, Bsplines.basis[, 4], type = "l", lty = 3, ylab = expression(paste(h[4](x))))
plot(x, Bsplines.basis[, 5], type = "l", lty = 3, ylab = expression(paste(h[5](x))))
plot(x, Bsplines.basis[, 6], type = "l", lty = 3, ylab = expression(paste(h[6](x))))
```



4.2.3 Natural Cubic Splines (NCS)

A cubic spline on $[a, b]$ is called a **natural cubic spline** if its second and third derivatives vanish at the endpoints a and b. Consequently the spline is linear on the two extreme intervals $[a, \xi_1]$ and $[\xi_m, b]$. The linearity constraints reduce the dimension of the space: a natural cubic spline with m interior knots has only m degrees of freedom (the usual textbook count).

Remark
If we place a knot at every distinct observation

$$x_i$$

and use a natural cubic spline, the resulting interpolant is a smooth curve that passes through every data point

$$(x_i, y_i)$$

.

Natural Cubic Spline

One convenient basis for the space of natural cubic splines with m interior knots is

$$\begin{aligned} N_1(x) &= 1 \\ N_2(x) &= x \\ N_{k+2}(x) &= d_k(x) - d_{k-1}(x) \end{aligned}$$

where

$$d_k(x) = \frac{(x-\xi_k)_+^3 - (x-\xi_m)_+^3}{\xi_m - \xi_k}$$

Each of these functions has vanishing second and third derivatives for $x \geq \xi_m$.

In R natural cubic splines are generated by the function `ns()` from the same `splines` package:

$$ns(x, df, knots, intercept = FALSE, Boundary.knots)$$

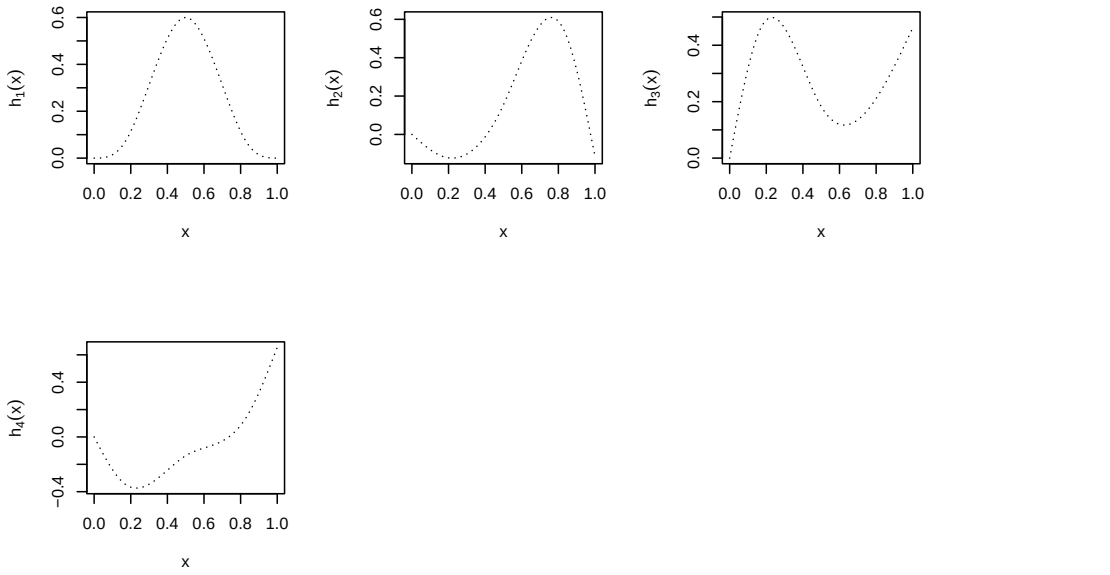
(Note that the default value of intercept is FALSE, exactly as for bs().)

Natural Cubic Splines in R

```
NCS.basis = ns(x, knots = new.knots, Boundary.knots = c(0, 1))
dim(NCS.basis)
```

```
## [1] 101   4
```

```
par(mfrow = c(2, 3))
plot(x, NCS.basis[, 1], type = "l", lty = 3, ylab = expression(paste(h[1](x))))
plot(x, NCS.basis[, 2], type = "l", lty = 3, ylab = expression(paste(h[2](x))))
plot(x, NCS.basis[, 3], type = "l", lty = 3, ylab = expression(paste(h[3](x))))
plot(x, NCS.basis[, 4], type = "l", lty = 3, ylab = expression(paste(h[4](x))))
```



Because the spline is forced to be linear outside the boundary knots, observations that fall outside those knots do not influence the fit. By default, therefore,

R places the boundary knots at the minimum and maximum of the observed

$$x_i$$

.

Knots versus degrees of freedom for `ns()`

- If interior knot locations are supplied, the number of columns of the basis matrix is

$$\text{length(knots)} + 1 + \text{as.integer(intercept)}$$

- If a target `df` is supplied, R places

$$m = \text{df} - 1 - \text{intercept}$$

interior knots at quantiles of x.

4.2.4 Regression Splines

For a fixed set of knots the corresponding cubic splines form a linear space of dimension

$$m + 4$$

. **Regression splines** simply expand the mean function in this basis:

$$g(x) = \beta_1 h_1(x) + \beta_2 h_2(x) + ... + \beta_p h_p(x)$$

- Ordinary Cubic Splines: $p = m + 4$.
- Natural Cubic Splines (NCS): $p = m$.

On the observed data the model may be written in matrix form

$$\begin{pmatrix} y_1 \\ y_2 \\ ... \\ y_n \end{pmatrix}_{n\times 1} = \begin{pmatrix} h_1(x_1) & ... & h_p(x_1) \\ h_1(x_2) & ... & h_p(x_2) \\ & & \\ h_1(x_n) & ... & h_p(x_n) \end{pmatrix}_{n\times p} \begin{pmatrix} \beta_1 \\ \beta_2 \\ ... \\ \beta_p \end{pmatrix}_{p\times 1}$$

where the $ n×p$ matrix $\mathbf{F}$ contains the basis functions evaluated at the x_i. The coefficient vector is obtained by ordinary least squares: Therefore, we can estimate the coefficients by solving the following Least-Squares problem:

$$\hat{} = \arg\min ||\mathbf{y} - \mathbf{F}\ ||^2$$

4.2.5 K-Fold Cross-Validation

A practical difficulty with regression splines is the choice of the number (and location) of knots, or equivalently the degrees of freedom. One systematic approach is

$$K$$

-fold cross-validation:

1. Fix a candidate number of knots (or degrees of freedom).

2. Randomly partition the data into K roughly equal folds.

3. For each fold $k = 1, \dots, K$:

   - hold fold k out as a validation set,
   - fit the regression spline on the remaining $K - 1$ folds,
   - compute the mean squared error MSE_k on the held-out fold.

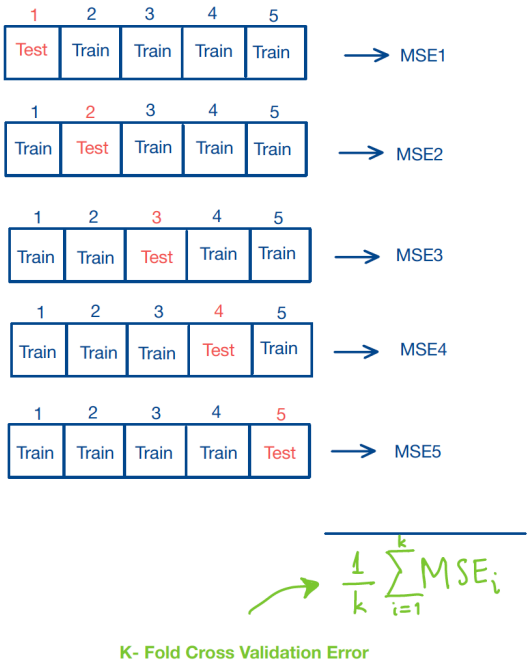
4. Average the K errors:

$$\mathrm{CV}(K) = \frac{1}{K} \sum_{k=1}^{K} \mathrm{MSE}_k.$$

5. Repeat the whole procedure for every candidate number of knots / degrees of freedom.

6. Select the value that minimises $\mathrm{CV}(K)$.

A schematic illustration of the procedure appears below.



4.3 Smoothing Splines

In regression splines we must choose both the number and the locations of the knots (or, equivalently, the degrees of freedom). As we have seen, B-splines and natural cubic splines both construct an $n \times p$ basis matrix $\mathbf{F}$ and then fit a linear model on that basis. Inevitably this requires decisions about the order of the spline, the number of knots (often guided by AIC, BIC or cross-validation), and even the placement of those knots—tasks that can be quite challenging. It is therefore natural to ask whether there exists an alternative approach that selects the effective complexity of the fit automatically.

A smoothing spline is designed precisely to balance fidelity to the data against smoothness of the fitted curve. The construction begins with an extreme idea: place a knot at every distinct observed value $x_1, \ldots, x_n$. This produces a natural-cubic-spline basis of dimension n,

$$\mathbf{y}_{n\times 1} = \mathbf{F}_{n\times p}\beta_{p\times 1}$$

Such a basis can interpolate the data perfectly and therefore overfits. To control the overfitting we borrow the idea of penalization introduced earlier and minimize a ridge-type objective

$$\min_{\beta}\Big\{||\mathbf{y} - \mathbf{F}\beta||^2 + \lambda\beta^T\Omega\beta\Big\}$$

where the tuning parameter $\lambda \geq 0$ is typically chosen by cross-validation and the penalty matrix Ω will be defined shortly. We now examine whether the solution of this penalized problem possesses attractive statistical properties.

4.3.1 The Roughness Penalty Approach

Let $S[a, b]$ be the space of all "smooth'' functions defined on $[a, b]$. This is a second order Sobolev space where global polynomial functions and cubic splines functions live ($S[a, b]$ is an infinite-dimensional function space. Our goal is to find the best function in $S[a, b]$ to approximate f.

Let us consider solving the following Penalized Residual Sum of Squares problem:

$$RSS_\lambda(g) = \sum_{i=1}^{n}[y_i - g(x_i)]^2 + \lambda \int_a^b [g''(x)]^2 dx.$$

where λ is a smoothing parameter. The first term measures the closeness of the model to the data, while the second term penalizes the roughness/curvature of the function. We solve this problem on $[a, b] = [\min x_i, \max_i]$ for functions with finite roughness penalty $\int_a^b [g''(x)]^2 dx < \infty$. This is known as a second order Sobolev space.

Note that $\int_a^b [g''(x)]^2 dx$ is called the **roughness penalty**.

From the expression above, we can see that λ is the smoothing parameter that controls the bias-variance trade-off. Therefore, when $\lambda = 0$, we interpolate the data and that leads us to overfitting. When $\lambda = \infty$, then we return to linear least-squares regression. It turns out that the solution to the penalized residual sum of squares has to be a NCS. Indeed,

Theorem

$$\min_g RSS_\lambda(g) = \min_{\tilde{g}} RSS_\lambda(\tilde{g})$$

} where $\tilde{g}$ is a NCS with knots at the unique n data points.

Let $g(x)$ be the optimal solution. Since the loss part in $RSS_\lambda(g)$ only involves n data points, we can find define a natural cubic spline (NCS) fit that we can call $\tilde{g}(x)$ such that it matches $g(x)$ at the observations x_i, $i = 1, \ldots, n$, i.e.

$$g(x_i) = \tilde{g}(x_i), i = 1, \ldots, n$$

We can always find such $\tilde{g}$ since our space consists of n basis. Then, we can show that

$$\int g''^2 dx \geq \int \tilde{g}''^2 dx$$

meaning that we will *always* prefer the $\tilde{g}$, the NCS "representation'' of g, since the *penalty* is smaller, and the *loss* doesn't change.

4.3.2 Proof

To establish the Theorem, we essentially need to show that

$$\int g''^2 dx \geq \int \tilde{g}''^2 dx$$

Thus, we define $h(x) = g(x) - \tilde{g}(x)$ for which we know that $h(x_i) = 0$ for $i = 1, \ldots, n$. Then

$$\int g''^2 \, dx = \int \tilde{g}''^2 \, dx + \int h''^2 \, dx + 2 \int \tilde{g}'' h'' \, dx$$

and without loss of generality assuming that the x_is are ordered, we obtain:

$$\begin{aligned}
\int \tilde{g}'' h'' \, dx &= \tilde{g}'' h' \Big|_a^b - \int_a^b h' \tilde{g}^{(3)} \, dx \\
&= -\sum_{i=1}^{n-1} \tilde{g}^{(3)}\left(x_j^+\right) \int_{x_j}^{x_{j+1}} h' \, dx \quad (\tilde{g}^{(3)} \text{ constant piecewise}) \\
&= -\sum_{i=1}^{n-1} \tilde{g}^{(3)}\left(x_j^+\right) \left(h(x_{j+1}) - h(x_j)\right)
\end{aligned}$$

The second equation is because $\tilde{g}$ is a NCS and therefore we know that it has zero second derivative on the two boundaries a and b. The third equation is true, because $\tilde{g}$ is at most a 3rd order polynomial on any region and as a consequence has constant third derivatives, which we can pull out of the integration. The last equation holds because we said that $h(x) = 0$ on all the observation points x_i. Therefore, this shows that the roughness penalty of our NCS solution is no larger than the best solution g. If we also take into account that $\tilde{g}$ is also in the space $S[a, b]$, then g must be our NCS solution.

4.4 Fitting Smoothing Splines

From the preceding discussion we conclude that the minimizer admits a finite-dimensional representation

$$\hat{g}(x) = \sum_i \beta_i N_i(x)$$

where the N_i are a natural-cubic-spline basis with knots placed at the unique observed values of $ x $. Substituting this expansion into the roughness penalty

yields

$$\begin{aligned}\int_a^b g''^2 dx &= \int \left(\sum_i \beta_i N_i''(x)\right)^2 dx \\ &= \sum_{i,j} \beta_i \beta_j \int N_i''(x) N_j''(x) dx \\ &= \beta^T \Omega \beta\end{aligned}$$

where the $n \times n$ penalty matrix has entries

$$\Omega_{ij} = \int N_i''(x) N_j''(x) dx$$

Consequently the penalized residual sum of squares becomes a function of the coefficient vector alone:

$$RSS_\lambda(\beta) = (\mathbf{y} - \mathbf{F}\beta)^T(\mathbf{y} - \mathbf{F}\beta) + \lambda\beta^T\Omega\beta$$

This is a ridge-type criterion whose minimizer is given in closed form by

$$\begin{aligned}\hat{\beta} &= \arg\min_\beta RSS_\lambda(\beta) \\ &= (\mathbf{F}^T\mathbf{F} + \lambda\Omega)^{-1}\mathbf{F}^T\mathbf{y}\end{aligned}$$

The smoothing spline version of the "hat'' matrix is called the **smoother matrix**

$$\hat{\mathbf{y}} = \mathbf{F}(\mathbf{F}^T\mathbf{F} + \lambda\Omega)^{-1}\mathbf{F}^T\mathbf{y} = S_\lambda \mathbf{y}$$

Remarks

We have done the analysis of degrees of freedom for ridge type regression. The degrees of freedom of a smoothing spline are

$$df = tr(S_\lambda)$$

which ranges between 0 and n. Under a special choice of basis (Demmler and Reinsch, 1975), a basis with double orthogonality property, can lead to

$$\mathbf{F}^T\mathbf{F} = \mathbf{I}, \Omega = diag(d_i)$$

where d_is are arranges in an increasing order and in addition $d_2 = d_1 = 0$. Using this basis we have

$$\hat{\beta} = (\mathbf{F}^T\mathbf{F} + \lambda\Omega)^{-1}\mathbf{F}^T\mathbf{y} = \Big(\mathbf{I} + \lambda diag(d_i)\Big)^{-1}\mathbf{F}^T\mathbf{y}$$

i.e.

$$\hat{\beta}_i = \frac{1}{1+\lambda d_i}\hat{\beta}_i^{LS}$$

Choosing λ

The smoothing parameter is selected by the same criteria used for ridge regression.

- Leave-one-out CV:

$$\text{CV}(\lambda) = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{g}(x_i)}{1 - S_\lambda(i,i)}\right)^2.$$

- Generalized CV:

$$\text{GCV}(\lambda) = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{g}(x_i)}{1 - \frac{1}{n}\text{tr}(S_\lambda)}\right)^2.$$

4.5 The `Birthrates` Example

In this chapter we relaxed the linearity assumption of classical regression by introducing polynomial regression, regression splines, and smoothing splines. All three approaches remain linear in the parameters, so they can still be fit by (penalized) least squares, yet they are flexible enough to capture nonlinear relationships. We now illustrate smoothing splines—the method that requires the least manual tuning of knots—with a classic data set on U.S. birth rates.

The analysis of this data set is available in R and Python.

Remark:

In the following Python file, you will also find the code for all the R examples illustrated in this chapter: Examples in Python